통합반 9회차 — 낭독 대본

[이론 01] 개별 작업을 하나의 비즈니스 프로세스로 관리하는 이유 이해하기

[SLIDE 01] AI와 사람이 함께 일하는 업무 프로세스의 설계와 실행

안녕하세요. 이번 회차에서는 AI와 사람이 함께 일하는 업무 프로세스를 다룹니다.

앞 회차에서는 에이전트의 판단과 그 판단을 검증하는 기준을 살펴봤습니다. 이번에는 추천안을 받은 뒤의 업무 전체로 시야를 넓혀 볼게요. 그런데 추천안이 나왔다고 설비가 바로 좋아질까요? 아니죠. 누군가 승인을 해야 하고, 명령이 설비로 가야 하고, 정말 나아졌는지 결과도 다시 봐야 합니다. 그럼 이 일들은 누가, 어떤 순서로 이어 가야 할까요?

그 순서와 책임을 프로세스로 정의하고, 누구나 같은 뜻으로 읽는 표준 그림으로 그려 볼 거예요. 그리고 그 그림을 직접 등록해서 실제로 실행까지 해 보겠습니다.

[SLIDE 02] 학습목표

이번 회차의 학습목표는 화면의 카드 네 장입니다.

첫 번째 카드는 비즈니스 프로세스와 BPM이에요. 작업 하나와, 시작부터 끝까지의 업무 전체를 구분하는 데서 출발합니다. 그 업무를 프로세스로 정의하려면 무엇을 정해야 하는지, 정의한 프로세스는 어떻게 고쳐 가는지도 보고요.

두 번째 카드는 BPMN 표기입니다. 프로세스를 BPMN이라는 표준 그림으로 읽는 법을 배워요. 승인과 거절이 어디서 갈리는지, 누가 어떤 일을 맡는지가 그림에 어떻게 나타나는지 따라가 볼 겁니다.

세 번째 카드는 프로세스 오케스트레이션이에요. 그린 프로세스가 실제로 어떻게 실행되는지를 봅니다. 지금 어디까지 왔는지, 결과가 다음 일로 어떻게 넘어가는지, 끝난 뒤에 무엇이 기록으로 남는지까지요.

마지막 카드가 실습입니다. 승인 절차와 긴급 대응 프로세스를 직접 그리고, 등록해서 실행한 다음, 처리 기록을 확인하겠습니다.

[SLIDE 03] 목차

목차를 보겠습니다. 이론은 여덟 단원인데, 크게 세 덩어리로 보시면 돼요. 위의 세 단원은 업무를 프로세스로 정의하고 관리하는 이야기입니다. 4번과 5번, 가운데 두 단원은 그 프로세스를 BPMN 그림으로 표현하는 방법이고요. 6번부터 8번까지는 그 그림이 실제로 실행되는 방식을 다룹니다.

맨 아래 9번 줄이 실습이에요. 네 가지가 이어집니다. 작은 승인 절차를 그리는 데서 시작해서, 긴급 대응 프로세스를 그리고, 그걸 실제 시스템에 연결한 다음, 쿨러 경보로 실행까지 해 보겠습니다.

이번 시간에는 이 가운데 위의 세 단원을 다룹니다. 추천안 뒤에 남은 일을 비즈니스 프로세스로 정의하고, 정의한 프로세스를 고쳐 가는 방법까지예요.

[SLIDE 04] 추천안 뒤에 남은 업무와 비즈니스 프로세스

첫 단원은 쿨러 사건으로 돌아가서, 추천안이 막 나온 시점에서 출발합니다. 기억나시죠? 유온이 오르고 냉각 효율이 떨어져서 쿨러 성능 저하 경보가 났고, 에이전트는 원인을 쿨러 핀 오염으로 가려서 팬 최대 운전을 추천했어요. 추천안이 나왔으니 일이 끝난 걸까요? 아직 아닙니다. 그렇다면 무엇이 남아 있고, 그 남은 일은 무엇이 이어 줄까요? 이 물음을 들고 시작해 보겠습니다.

[SLIDE 05] 추천안 뒤에 남은 다섯 가지 일

화면을 보시면, 쿨러 사건이 처음부터 끝까지 지나가는 자리를 한 줄로 늘어놓았어요. 그런데 진하게 그린 건 왼쪽의 경보와 추천, 딱 두 칸뿐이죠. 추천안이 나온 시점에는 여기까지만 끝난 겁니다.

그 순간 설비는 어떤 상태일까요? 이 추천에 따른 냉각 명령은 아직 나가지 않았습니다. 추천안을 만들었다는 사실만으로 유온이 회복됐다고 볼 수는 없어요. 실제로 쿨러 사건에서는 운전원이 승인하던 순간의 유온이, 경보가 났을 때보다 오히려 더 올라 있었어요. 추천안이 화면에 머무는 동안에도 설비는 기다려 주지 않는다는 거죠.

회색으로 남은 칸을 오른쪽으로 읽어 볼게요. 먼저 운전원이 추천안을 승인해야 합니다. 승인된 조치는 냉각 명령으로 설비에 가야 하고요. 명령 뒤에는 유온을 다시 읽어서 정말 내려왔는지 확인해야 하는데, 이걸 재관측이라고 합니다. 그런데 팬을 최대로 돌리는 건, 1회차에서 본 완화 조치예요. 유온은 당장 낮추지만 핀에 낀 오염은 그대로 남죠. 그래서 근본 조치인 핀 세척은 작업지시로 따로 남겨야 합니다. 그리고 마지막으로, 무엇을 했고 결과가 어땠는지 보고합니다.

에이전트가 한 일은 이 줄의 앞쪽 일부예요. 판단이 아무리 정확해도, 뒤의 실행과 확인이 이어져야 원하는 결과를 얻었는지 알 수 있습니다.

[SLIDE 06] 남은 일을 맡는 쪽과 순서

이 다섯 가지는 누가 맡을까요? 화면의 다섯 칸에서 맨 윗줄을 보세요. 그게 그 일을 맡는 쪽이에요. 사람과 여러 시스템이 나눠 맡고, 같은 시스템이 두 일을 맡기도 하죠.

첫 칸, 승인은 운전원이 맡습니다. 추천안을 검토해서 승인하거나 거절하죠.

두 번째 칸, 냉각 명령은 SCADA가 맡아요. 승인된 조치를 PLC에 보내서 팬 속도를 바꿉니다.

세 번째 칸 재관측은 조금 다릅니다. 처리 전체를 이끄는 시스템이 직접 하거든요. 이 시스템이 정확히 무엇인지는 실행을 다룰 때 다시 만나게 될 거예요. HYD의 이 경보에서는 유온이 55도 아래

로 내려왔는지와 경보 해제 수신을 함께 확인합니다. 여기서 55도는 이 경보에 정한 회복 기준이에요.

명령이 나갔으면 됐지, 왜 또 확인할까요? 1회차 복습에서 명령 처리 기록과 완화 확인 기록을 구별했던 것 기억나시죠. 설비가 명령을 실행했다고 답한 것과, 유온이 실제로 내려온 것은 서로 다른 사실이었어요. 명령은 처리됐는데 냉각 효과는 얻지 못할 수도 있고요. 그래서 명령 다음에, 효과를 따로 확인하는 일을 둡니다.

네 번째 칸 작업지시 등록은 CMMS가 맡습니다. 설비 보전 관리 시스템이라는 뜻인데, 핀 세척 같은 정비 작업을 작업지시로 남기는 곳이에요. 마지막 칸 결과 보고는 다시 처리를 이끄는 시스템이 맡고요.

왼쪽에서 오른쪽으로 가는 순서도 같이 보세요. 승인이 나야 명령을 보낼 수 있고, 명령이 끝나야 재관측을 시작할 수 있습니다. 다섯 가지 모두 앞의 결과를 받아야 다음이 시작되는 거죠.

[SLIDE 07] 일과 일 사이를 잇는 것이 없을 때

맡을 쪽은 이렇게 다 있어요. 그런데 이 다섯 쪽 사이를 이어 주는 게 없다면 어떻게 될까요? 화면의 카드 세 장이 바로 그때 생기는 문제입니다.

첫 번째 카드, 흐름이 멈춥니다. 추천안이 화면에 떠 있어도 운전원이 그걸 모르면 승인은 일어나지 않아요. 재관측도 누군가 기억해서 챙겨야만 이루어지고요. 쿨러 사건에서 재관측은 명령 뒤에 한동안 유온을 지켜봐야 끝나는 일이었어요. 경보가 날 때마다 사람이 이걸 끝까지 지키고 있기는 쉽지 않죠.

두 번째 카드, 책임이 흩어집니다. 누가 승인해야 하는지, 운전원이 자리에 없어서 응답이 없으면 어떻게 할지가 정해져 있지 않아요. 그러면 다들 누군가 처리했겠지 하고 넘어가기 쉽습니다.

세 번째 카드는 기록이 남지 않는다는 거예요. 이 사건이 지금 어디까지 왔는지, 결국 무엇으로 끝났는지를 한곳에서 볼 수가 없습니다. 나중에 왜 그렇게 처리했는지 확인하려면, 사람들한테 하나하나 물어봐야 하죠.

그러니까 일 하나하나를 잘하는 것과, 일 전체가 끝까지 가는 것은 서로 다른 문제입니다.

[SLIDE 08] 에이전트의 판단과 정해 두어야 하는 순서

그렇다면 이 연결을, 판단을 잘하는 에이전트한테 맡기면 되지 않을까요? 충분히 그런 생각이 들 수 있어요. 화면 왼쪽은 에이전트가 하는 일, 오른쪽은 정해 두어야 하는 일입니다. 두 칸은 성격이 다릅니다.

왼쪽부터 볼게요. 에이전트는 지식과 값을 읽어서 원인을 가리고, 조치 후보를 견주어 하나를 추천합니다. 상황을 읽고 그때마다 판단하는 데 강하죠. 다만 같은 입력을 줘도 거치는 과정이나 표현은 달라질 수 있어요. 실제로 같은 지침을 받은 에이전트도, 처리 건에 따라 도구를 부르는 순서가 지침에 적힌 순서와 달랐던 적이 있습니다. 판단에 걸리는 시간도 매번 다르고요.

이제 오른쪽 칸입니다. 승인 전에는 설비 명령이 나가지 않는다. 승인은 정해진 역할만 한다. 명령 뒤에는 반드시 유온을 다시 읽는다. 이게 그때그때 판단할 일일까요? 아니죠. 매번 똑같이 지켜져야 하고, 한 번이라도 빠지면 안 되는 약속이에요.

판단의 과정은 상황에 따라 달라질 수 있지만, 정해 둔 권한과 선행 조건은 매번 지켜져야 합니다. 승인 전에는 명령이 없다는 약속이 어느 날은 지켜지고 어느 날은 빠진다면, 그런 시스템에 설비를 맡길 수는 없으니까요. 그래서 HYD는 지침에 적는 데 더해 프로세스와 서버의 검사로 이 약속을 지킵니다. 어떤 제품을 쓰든 지침과 실제 강제 장치를 함께 확인해야 해요.

[SLIDE 09] 비즈니스 프로세스와 처리 건

순서와 책임을 미리 정해 둔 그 틀, 그게 비즈니스 프로세스입니다. 원하는 결과를 내기 위해 관련된 작업과 그 흐름을 정한 묶음이에요. 이 사례에서는 여러 주체가 일을 나눠 맡습니다.

첫 줄의 정의에서 두 가지만 짚을게요. 시작하는 사건이 있고, 끝났다고 보는 조건이 있습니다. 그러니까 작업을 그냥 늘어놓은 목록이 아니라, 무엇으로 시작해서 무엇으로 끝나는지가 정해진 묶음이라는 거죠.

둘째 줄은 우리 예입니다. 쿨러 성능 저하 경보로 시작해서 결과 보고로 끝나는 긴급 대응 프로세스예요. 에이전트의 추천안, 운전원의 승인, 시스템의 명령과 기록이 이 안에서 한 순서로 이어집니다.

셋째 줄은 처리 건이에요. 이 구분이 중요합니다. 프로세스는 경보가 올 때마다 쓰는 틀이고요, 경보 한 건에 대해 그 틀이 실제로 한 번 수행되는 것을 처리 건이라고 부릅니다. 쿨러 사건도 긴급 대응 프로세스의 처리 건 하나로 진행됐어요.

이렇게 틀과 건을 나눠 두면 좋은 점이 있어요. 틀은 하나라서, 정의와 실행 장치를 함께 마련하면 경보마다 정해 둔 조건과 책임에 따라 진행할 수 있습니다. 그리고 건은 경보마다 따로 열리니까, 이 경보는 지금 어디까지 왔고 무엇으로 끝났는지를 건마다 따로 볼 수 있죠.

[SLIDE 10] Task와 End-to-End

그 처리 건 하나가 지나간 자리를, 앞에서 본 한 줄 그림에 다시 놓아 봤습니다. 이번에는 괄호 두 개를 보세요. 위쪽의 짧은 괄호는 추천 하나만 묶고, 아래쪽의 긴 괄호는 경보부터 끝까지 전체를 묶어요. 그리고 앞 장에서 회색이던 다섯 칸이, 이번에는 모두 진하게 칠해져 있죠. 처리 건 하나가 이 칸들을 실제로 다 지나갔다는 뜻이에요.

위쪽처럼 프로세스 안의 작업 하나를 Task라고 합니다. 에이전트의 원인 진단과 조치 제안이 Task 하나예요.

아래쪽처럼 시작 사건에서 최종 결과까지 전체를 묶은 건 End-to-End라고 하고요. 우리말로는 종단 간이라고 합니다.

쿨러 사건에서 두 괄호가 끝난 시점을 비교해 볼까요? 괄호마다 붙은 끝난 시점 줄을 보세요. 위쪽, 에이전트의 Task가 끝났을 때는 설비 명령이 하나도 없었어요. 그 추천에 따른 완화 효과는 아직 확인하지 못했고요. 아래쪽, End-to-End가 끝났을 때는 다릅니다. 팬 속도를 올리는 명령이 나갔고, 유온은 회복 기준 아래로 내려왔고, 핀 세척 작업지시도 남았죠.

시간으로 보면 더 분명해요. 위쪽 괄호에는 걸린 시간이 처리 건 전체의 절반 남짓이라고 적혀 있죠. 에이전트의 Task가 처리 건 시간의 절반 넘게를 썼는데도, 그 시점에는 추천 뒤의 실행과 효과 확인이 남아 있었던 겁니다.

그래서 업무가 끝났는지는 Task가 아니라 End-to-End로 판단합니다. 에이전트가 자기 Task를 잘 끝냈다는 건, 이 업무의 앞부분을 잘 해냈다는 뜻이에요. 추천안이 나왔는데도 일이 끝나지 않았던 까닭이 바로 여기 있습니다.

여기서 전체의 경계도 확인해야 합니다. 이 처리 건은 핀 세척 작업지시를 등록하고 보고하는 데서 끝나요. 실제 세척이 완료된 것까지 뜻하지는 않습니다. 개선 시간을 비교할 때도 시작과 끝을 같게 정해야, 한 작업이 빨라진 것을 전체 업무가 좋아진 것으로 오해하지 않습니다.

[SLIDE 11] SOP와 프로세스의 범위

순서를 정한다는 점에서 SOP와 프로세스는 닮았습니다. 앞에서 배운 팬 최대 운전의 SOP는 그 조치를 어떤 조건과 단계로 수행하고 완료를 확인할지 정했죠.

화면 왼쪽은 그 조치의 절차이고, 오른쪽은 경보에서 승인, 실행, 확인, 보고까지 여러 작업을 잇는 업무 흐름입니다. 이 수업에서는 이 두 범위를 비교합니다.

다만 SOP가 언제나 작은 조치 하나만 다루는 것은 아닙니다. 조직에 따라 넓은 업무 절차를 SOP로 문서화할 수도 있어요. 여기서는 이름보다 역할을 보시면 됩니다. 조치를 수행할 기준이 무엇이고, 그 조치가 언제 누구의 결정으로 실행되는지를 서로 맞추는 것이 중요합니다. 프로세스의 승인 작업에서는 바로 그 SOP로 정해 둔 조치를 검토합니다.

[SLIDE 12] 프로세스를 정의하는 다섯 가지

그 프로세스를, 두 번째 단원에서는 실제로 세워 봅니다. 비즈니스 프로세스를 하나 세운다는 건, 실제로 뭘 정하는 일일까요? 긴급 대응 프로세스에 대어 보면서 다섯 가지를 차례로 정해 보겠습니다.

[SLIDE 13] 다섯 가지와 긴급 대응 프로세스의 답

이번 수업에서는 프로세스를 정의할 때 먼저 정할 것을 다섯 가지로 묶었습니다. 표의 가운데 열에 있는 물음을 하나씩 읽어 볼게요.

무엇이 일어나면 시작하는가, 이게 시작 조건입니다. 시작할 때 무엇을 받는가는 입력이고, 끝났을 때 무엇이 남는가는 출력이에요. 작업마다 누가 맡는가는 책임자, 그리고 무엇으로 끝났다고 보는가가 완료 조건입니다.

오른쪽 열은 긴급 대응 프로세스의 답이에요. 쿨러 성능 저하 경보가 오면 시작하고, 설비와 경보의 내용을 받아서, 결과 보고를 남기고, 정해진 조건을 충족한 경로에서는 작업지시도 남깁니다. 작업은 에이전트, 운전원, 시스템이 나눠 맡고요. 그리고 진행해야 할 흐름이 정해 둔 끝까지 마무리됐는지 확인합니다. 한 갈래만 끝나고 다른 일이 남아 있으면 전체는 아직 진행 중이에요. 끝났다는 상태와 원하는 결과를 얻었다는 판단도 구별해야 합니다. 다음 사례에서 그 차이를 보겠습니다.

여기서 기억해 둘 게 있어요. 이 다섯 가지는 실행하기 전에 정해 둬야 한다는 겁니다. 일이 벌어진 뒤에 정하면, 앞에서 본 멈춤이나 흩어진 책임이 그대로 생기니까요.

[SLIDE 14] 시작 조건과 입력: 경보 메시지 한 건

다섯 가지를 하나씩 자세히 보겠습니다. 첫째와 둘째, 시작 조건과 입력이에요. 화면 왼쪽이 쿨러 사건의 경보 메시지입니다. 1회차에서 본 바로 그 경보예요.

이 프로세스는 사람이 조작해서 여는 게 아닙니다. 경보 메시지가 도착하는 것, 그게 시작 조건이에요. 오른쪽 첫 줄을 보시면, 경보의 종류를 나타내는 칸이 쿨러 성능 저하인 경보가 오면 긴급 대응 프로세스가 열립니다.

실제로 쿨러 사건에서도 경보가 도착한 뒤 처리 건이 열렸어요. 둘째 줄을 보세요. 어느 설비인지, 어느 경보인지를 가리키는 식별자가 처리 건의 입력으로 실립니다. 에이전트도 이 입력에서 설비와 경보를 확인하고 진단을 시작하죠.

그런데 입력에 식별자만 실리는 건 아니에요. 경보 메시지 전체도 함께 실립니다. 그래서 뒤의 작업들은 경보가 났을 때의 내용을 다시 찾으러 가지 않아요. 처리 건이 받은 같은 입력을 읽습니다. 출발점의 경보 내용을 함께 참조하기 위한 겁니다. 이후의 현재값은 다시 조회할 수 있으므로, 시작 입력과 나중 관측을 같은 시점의 값으로 보지는 않습니다.

마지막 줄은 약속 하나예요. 같은 경보로 처리 건이 두 번 열리지 않는다는 겁니다. 1회차에서 메시지가 전달되는 길을 볼 때, 같은 메시지가 다시 도착할 수 있으니 받는 쪽이 중복을 가려야 한다고 했었죠. 프로세스도 경보를 받는 쪽이에요. HYD는 같은 운영 영역에서 같은 정의와 사건 식별자로 이미 열린 처리 건이 있으면 새로 열지 않습니다. 중복 전달 때문에 업무가 다시 시작되는 일을 막는 장치예요. 다만 시작의 중복 방지만으로 모든 외부 명령이 정확히 한 번 실행된다고 보장할 수는 없습니다. 명령을 받는 쪽의 중복 검사와 처리 결과 확인도 필요합니다.

[SLIDE 15] 출력과 입력의 연쇄

셋째는 출력입니다. 프로세스 전체의 출력은 결과 보고와 작업지시였죠. 그런데 작업 하나하나에도 받는 것과 내는 것이 정해져 있어요.

화면 왼쪽 칸부터 읽어 볼게요. 원인 진단과 냉각 조치 제안은 경보를 받아서 원인과 추천안을 냅니다. 운전원 승인은 그 추천안을 받아서 승인이나 거절을 내고요. 냉각 명령은 승인된 조치를 받아 명령의 처리 결과를 내고, 재관측은 명령이 끝났다는 걸 받아서 회복했는지를 냅니다.

보이시죠? 앞 칸이 낸 게 다음 칸이 받는 게 됩니다. 출력과 입력이 사슬처럼 이어지는 거예요.

그래서 작업마다 무엇을 내야 하는지를 미리 정해 둬야 합니다. 실제로 실행할 때도, 작업마다 내겠다고 정해 둔 값만 다음으로 넘어가요. 이것은 HYD의 변수 전달 규칙입니다. 다른 시스템에서도 전달할 항목과 의미를 명확히 해야 한다는 원리는 같아요. 예를 들어 에이전트가 추천안을 내지 못하면 어떻게 될까요? 운전원은 승인할 게 없고, 흐름은 그 자리에서 멈춰 버리죠.

[SLIDE 16] 책임자: 사람, 에이전트, 시스템의 역할

넷째는 책임자입니다. 여기서는 작업을 맡는 수행 주체를 말해요. 화면의 세 카드로 나눠 보겠습니다.

첫째, 담당을 역할로 정합니다. 특정 사람 이름 대신 운전원 역할이 승인한다고 정하면, 교대할 때 흐름 전체를 다시 그릴 필요는 없어요. 대신 실제 구성원과 작업 배정은 바뀐 근무 상황에 맞춰 관리해야 합니다.

조치별로 필요한 승인 역할도 다를 수 있습니다. 이 사례에서 팬 최대 운전의 지정 역할은 운전원이고, 핀 세척은 설비보전팀장이에요. HYD의 조치 승인 검사는 지정 역할 또는 그보다 엄격히 높은 등급을 허용합니다. 같은 등급의 다른 역할이 자동으로 허용되는 것은 아니고, 사람 계정이 그 역할의 구성원인지도 확인합니다. 작업을 배정받는 것과 조치를 승인할 권한이 있는 것은 함께 확인할 조건입니다.

둘째, 수행 주체에는 에이전트와 시스템도 있습니다. 이 흐름은 추천, 사람의 결정, 시스템 실행을 나누어 배치했어요. 에이전트가 작업을 맡는다고 해서 조직의 최종 책임까지 기계로 넘어가는 것은 아닙니다. 기준과 권한을 정하고 운영 결과를 책임질 사람도 있어야 해요.

셋째, 각 작업에서 누구의 응답을 기다리는지 분명히 합니다. 이 예에서는 작업마다 수행 주체를 정해, 지금 누구 차례인지와 어디서 지연되는지를 찾을 수 있게 했습니다.

[SLIDE 17] 완료 조건: 프로세스가 닫히는 경우

다섯째는 완료 조건이에요. 언제 끝났다고 볼까요? 잘 끝난 경우만 떠올리기 쉬운데, 표를 보면 경우가 네 가지입니다.

첫 줄은 승인했고 유온도 회복한 경우예요. 작업지시를 등록하고, 정상이라는 결과 보고로 닫힙니다. 쿨러 사건이 이렇게 끝났죠.

둘째 줄은 승인해서 명령은 나갔는데, 유온이 기준까지 안 내려온 경우입니다. 이때도 열어 두지 않아요. 미달이라는 결과 보고로 닫습니다. 회복하지 못했다는 것도 하나의 결과거든요. 그 결과가 보고로 남아야, 사람이 다음에 무엇을 할지 판단할 수 있죠.

셋째 줄은 운전원이 거절한 경우예요. 오른쪽 열을 보시면 설비 명령이 없죠. 명령은 하나도 나가지 않고, 명령을 내지 않았다는 거절 보고로 닫힙니다. 거절도 업무의 한 결말이라 이렇게 기록에

남겨요. 보고는 결말을 요약하고, 실제 명령 여부를 확인할 때는 명령 단계와 실행 기록도 함께 대조합니다.

넷째 줄은 운전원이 아예 응답하지 않는 경우입니다. 이때는 닫지 않아요. 설정한 업무 시간 10분이 지나면 지연 알림만 보내고 승인을 계속 기다립니다. 이 실습 환경은 20배속이어서 실제 대기는 30초입니다.

이 마지막 줄은 꼭 기억해 두세요. 시간이 지났다고 자동으로 승인하거나 자동으로 거절하지 않습니다. 사람의 결정이 필요한 자리는 사람이 결정할 때까지 기다리게 정해 둔 거예요. 만약 정상인 경우만 정해 뒀다면 어떨까요? 나머지 경우의 업무는 끝나지도 않고, 기록도 남지 않은 채 떠 있게 되겠죠.

미달이나 거절로 처리 건이 닫혀도 설비 문제가 해결됐다는 뜻은 아닙니다. 그 뒤의 현장 확인이나 재요청이 필요하면 맡을 사람과 후속 절차를 별도로 정해야 합니다. 완료 조건은 일을 끝내기 위한 표시와 결과에 대한 책임을 함께 정하는 자리예요.

[SLIDE 18] 프로세스가 에이전트 밖에서 보장하는 것

첫 단원에서 남겨 둔 물음이 하나 있었죠. 에이전트에게 맡길 수 없다고 한 순서와 책임, 프로세스는 이걸 실제로 어떻게 지킬까요?

첫 번째 카드는 순서입니다. 냉각 명령 작업은 승인 작업이 승인으로 끝난 뒤에만 열려요. 추천안만으로는 명령이 만들어지지 않습니다. 한 발 더 나가서, 승인 단계가 빠진 흐름은 실행할 흐름으로 시스템에 올리는 단계에서부터 받아 주지 않아요. 승인 단계를 넣었더라도, 거절 쪽 갈래에서 설비에 효과가 나가게 그린 흐름도 마찬가지고요. 승인 없이 설비에 효과를 내는 흐름은 처음부터 실행될 수 없게 막아 둔 거죠. 이 검사는 셋째 실습에서 직접 만나게 됩니다.

두 번째 카드는 책임이에요. 작업의 담당과 조치의 승인 권한을 확인합니다. 이 예의 지정 역할은 운전원이지만, 앞에서 본 역할 등급과 구성원 조건도 적용돼요. 권한을 충족하지 못한 승인은 거부되고 기록으로 남습니다.

세 번째 카드는 기록입니다. 어느 경보로 열렸고, 누가 승인했고, 어디서 끝났는지가 처리 건에 남아요. 그래서 나중에 되짚어 볼 수 있죠.

에이전트 쪽에도 같은 경계가 있어요. 이 냉각 대응에서 연결한 업무 도구는 조회와 판단 제출을 중심으로 제한했습니다. 에이전트가 일반적으로 읽기만 할 수 있다는 뜻은 아닙니다. 제출한 판단에 설비 명령 같은 내용을 담으려 해도 받아들여지지 않아요. 명령은 승인을 거친 뒤에, 프로세스 쪽에서만 만들어집니다.

그러니까 에이전트가 아무리 확신에 찬 추천을 내도 이 순서는 바뀌지 않아요. 프로세스의 정의와 등록 검사, 실행 시 권한·조건 검사가 함께 적용되기 때문입니다. 그림에 승인 상자 하나를 넣는 것만으로 보장되는 일은 아니에요.

[SLIDE 19] 지식 그래프에 등록된 프로세스

프로세스를 정의할 때 하나 더 생각해 볼 게 있어요. 이 업무는 대체 무엇을 위해 있는가, 하는 겁니다. 지식 그래프에는 그 답도 들어 있어요.

화면 가운데 노드가 설비 이상 조치라는 프로세스입니다. 지식 그래프에는 프로세스라는 클래스가 있고, 설비 이상 조치는 그 인스턴스예요. 우리가 따라온 긴급 대응 프로세스와 이름이 다르죠? 이 노드는 긴급 대응 프로세스를 가리키는 게 아니라, 지식 그래프에 원래 올라 있는 설비 이상 대응 업무예요. 여기서는 지식 그래프가 프로세스의 목적을 어떻게 적어 두는지, 그 방식만 보겠습니다.

왼쪽으로 가는 관계는 이 프로세스가 어느 설비를 대상으로 도는지를 나타냅니다. 오른쪽으로 뻗은 세 관계는 이 프로세스가 달성하려는 전략 목표를 가리키고요. 설비 가용성 확보, 무재해 운전, 납기 신뢰. 성과를 공부할 때 본 그 전략 목표들이죠.

그래서 설비 이상에 대응하는 업무가 왜 있는지를 조회로 확인할 수 있어요. 설비를 계속 쓸 수 있게 하고, 사고 없이 운전하고, 납기를 지키기 위해서죠.

목적이 적혀 있으면, 나중에 이 업무를 고칠 때 무엇을 지켜야 하는지가 분명해집니다. 처리를 빨리 끝내려고 승인 단계를 빼자는 의견이 나와도, 무재해 운전이라는 목표에 비춰 보면 받아들일 수 없는 거죠. 이렇게 시작 조건, 입력, 출력, 책임자, 완료 조건을 정하고 그 목적까지 적어 두면, 프로세스 하나가 세워지는 겁니다.

[SLIDE 20] BPM의 생애주기와 AS-IS · TO-BE

그런데 프로세스는 한 번 세우면 끝일까요? 목표나 설비나 조직이 바뀌는데 프로세스만 그대로 둘 수는 없겠죠. 그래서 세 번째 단원에서는 세운 프로세스를 고쳐 가는 방법을 봅니다. 정의한 프로세스를 바뀌는 환경에 맞춰 계속 고쳐 가려면, 어떤 방법이 필요할까요? 그리고 고칠 곳은 어떻게 찾아낼까요? 이 두 물음을 차례로 풀어 보겠습니다.

[SLIDE 21] BPM: 프로세스를 관리하는 방법

먼저 방법부터요. 프로세스를 만들고, 운영하고, 고쳐 가는 일 전체를 관리하는 방법, 이걸 업무 프로세스 관리, BPM이라고 합니다.

첫 줄의 정의를 보면 분석하고, 설계하고, 실행하고, 결과를 보며 고쳐 간다고 돼 있죠. 그런데 핵심은 그 아래 문장이에요. 프로세스를 한 번 만들고 끝내는 게 아니라, 계속 다듬어 가는 대상으로 본다는 겁니다.

둘째 줄은 BPM이 무엇을 다루느냐예요. 작업 하나를 얼마나 빨리 하느냐가 아니라, 시작 사건에서 최종 결과까지의 흐름 전체입니다.

앞에서 구분한 End-to-End가 여기서 관리의 단위가 되는 거예요. 왜 흐름 전체를 볼까요? 셋째 줄을 보세요. 에이전트의 판단 시간이 줄어도 승인 대기가 대부분이라면 전체 처리 시간의 개선은 작을 수 있잖아요. 그래서 흐름 전체를 놓고 어디를 고칠지 봐야 합니다.

[SLIDE 22] BPM의 생애주기: 다섯 단계의 되풀이

여기서는 BPM의 활동을 분석, 설계, 실행, 모니터링, 개선의 다섯 단계로 묶어 봅니다. 자료마다 단계의 이름과 묶음은 조금 달라도, 실행 결과를 다음 개선에 반영한다는 원리는 같습니다. 위쪽 가운데의 분석에서 시작해서 시계 방향으로 돌아 볼게요.

단계마다 붙은 그림을 보면 하는 일이 짐작되실 거예요. 출발점의 돋보기가 분석입니다. 지금의 업무를 조사하는 단계죠. 오른쪽 위의 흐름 그림은 설계, 바꿀 흐름을 그리는 단계고요. 오른쪽 아래 재생 표시는 실행, 그린 대로 실제로 수행하는 단계입니다. 왼쪽 아래 관측 화면은 모니터링, 그 결과를 지켜보는 단계예요. 그리고 왼쪽 위의 오르는 막대가 개선, 지켜보면서 찾은 문제를 고치는 단계입니다.

화살표를 잘 보시면, 왼쪽 위 개선에서 분석으로 돌아가는 화살표만 파랗게 그려져 있죠. 개선에서 고친 내용이 다시 분석의 대상이 되기 때문입니다.

그래서 직선이 아니라 고리 모양이에요. 끝없이 돈다는 뜻이 아니라, 업무와 환경이 바뀌는 한 계속 돌아간다는 뜻입니다.

[SLIDE 23] 다섯 단계에서 하는 일

각 단계에서 실제로 뭘 하는지, 쿨러 사건에 대어 볼게요. 표의 왼쪽은 단계, 가운데는 그 단계에서 하는 일, 오른쪽은 쿨러 사건에서 그 단계에 해당하는 거예요.

분석에서는 지금의 업무를 읽고 시작 조건, 책임자, 완료 조건을 가려냅니다. 쿨러 사건이라면 쿨러 성능 저하 경보가 업무를 열고, 운전원이 승인한다는 걸 찾아내는 일이죠. 앞 단원에서 정한 다섯 가지도 이렇게 지금의 업무를 읽는 데서 나옵니다. 그리고 일이 어디서 끊기는지도 이때 함께 찾아요.

설계에서는 바꾸려는 흐름을 그림으로 그려요. 경보, 추천, 승인, 명령, 재관측, 보고의 순서를 그리는 거죠.

실행에서는 그 그림을 등록하고 배포해서 실제 경보에 쓰이게 합니다. 경보가 한 건 오면, 처리 건이 하나 열려요.

모니터링에서는 처리 건마다 누가 맡았고, 어디서 기다렸고, 단계마다 시간이 얼마나 걸렸고, 결과가 어땠는지를 지켜봅니다. 운전원 승인에서 얼마나 기다렸는지, 재관측 결과가 어땠는지 같은 것들이요. 처리 건의 기록에 이 값들이 단계마다 남아 있어서, 그걸 읽는 일이 모니터링이에요.

마지막 개선에서는 그렇게 찾은 문제를 반영해서 흐름을 고치고 다시 등록합니다. 여기서 하나 짚어 둘게요. HYD에서 고친 흐름을 배포하면 다음 경보의 처리 건부터 쓰입니다. 이미 진행 중인 건까지 바뀌지는 않아요. 이 이야기는 마지막 단원에서 판본이라는 개념으로 다시 보겠습니다.

오래 걸린 단계를 곧바로 없애야 할 단계라고 판단하면 안 됩니다. 효과를 관측하는 데 필요한 시간인지, 담당자가 몰라서 기다린 시간인지 먼저 구별해야 해요. 같은 종류의 여러 처리 건에서 지연 사유와 결과를 비교하고, 필요한 확인은 유지하면서 불필요한 대기를 줄이는 것이 개선입니다.

[SLIDE 24] AS-IS와 TO-BE: 지금의 방식과 바꾸려는 방식

분석과 설계에서 자주 쓰는 낱말이 두 개 있어요. AS-IS와 TO-BE입니다. AS-IS는 지금 일하는 방식을 있는 그대로 그린 것이고, TO-BE는 바꾸려는 방식을 그린 거예요.

위 줄이 AS-IS의 예입니다. 미리 말씀드리면, 어느 현장의 실제 과거를 그린 건 아니에요. 프로세스가 정해져 있지 않은 곳에서 흔히 볼 수 있는 모습을 일반적인 예로 놓은 겁니다. 경보를 본 운전원이 전화를 걸어서 보전 담당을 찾죠. 그런데 마지막 선이 흐릿하게 그려져 있어요. 그다음에 무슨 일이 일어났는지가 분명하지 않다는 뜻입니다.

아래 줄은 같은 업무를 프로세스로 옮긴 TO-BE예요. 경보가 오면 추천이 나오고, 승인을 거쳐서 명령으로 이어집니다. 도형의 뜻은 다음 단원에서 배울 거니까, 지금은 순서와 갈림이 그림에 분명히 드러난다는 것만 봐 두세요.

지금 방식이 엉성해 보여도, AS-IS는 있는 그대로 그려야 해요. 어디서 끊기는지는 그려 봐야 보이고, 바꾼 뒤에 무엇이 나아졌는지도 이 그림과 견줘야 말할 수 있으니까요. 이렇게 분석은 AS-IS를 그려서 문제를 찾는 일이고, 설계는 TO-BE를 그리는 일입니다.

[SLIDE 25] AS-IS와 TO-BE를 견주어 얻는 것

두 그림을 나란히 놓으면 무엇이 바뀌는지가 분명해져요. 왼쪽과 오른쪽에서 같은 줄끼리 견주어 보세요.

왼쪽 AS-IS의 세 줄, 어디서 본 것 같지 않으세요? 첫 단원에서 본 세 문제예요. 흐름이 멈추고, 책임이 흩어지고, 기록이 남지 않는다고 했었죠. 그 세 문제가 AS-IS 그림에서는 전화로 담당을 찾는 자리, 확인을 각자 기억하는 자리, 결정을 물어봐야 아는 자리로 보입니다.

오른쪽 TO-BE는 그 자리마다 무엇을 놓았는지를 보여 줘요. 담당을 찾던 자리에는 운전원 앞에 놓이는 승인 작업이, 기억에 맡기던 확인 자리에는 정해진 순서로 열리는 명령과 재관측이, 물어봐야 알던 결정 자리에는 처리 건에 남는 기록이 들어갔습니다.

보시면, AS-IS에서 사람이 기억과 전화로 잇던 자리가 TO-BE에서는 작업과 순서와 기록으로 바뀌었죠. 그 자리를 찾아내는 게 분석의 결과입니다. 고칠 곳은 이렇게 두 그림을 견주어서 찾고, 찾은 건 BPM의 고리를 따라 설계하고, 실행하고, 다시 지켜봅니다.

그림을 바꿨다는 사실만으로 개선을 입증한 것은 아닙니다. 같은 범위의 업무에서 누락된 확인이 줄었는지, 대기와 재작업이 줄었는지, 정한 결과를 얻었는지를 운영 기록으로 비교해야 합니다.

다음 시간에는 이 TO-BE를 누구나 같은 뜻으로 읽는 그림, BPMN으로 그리는 법을 보겠습니다.

[이론 02] 복잡한 업무를 공통된 그림으로 표현하는 방법 이해하기

[SLIDE 26] BPMN: 이벤트 · 작업 · 게이트웨이

이번 시간에는 업무를 BPMN이라는 표준 그림으로 표현하는 방법을 봅니다. 앞 시간에 BPM의 설계는 바꾸려는 방식, TO-BE를 그리는 일이라고 했죠. 그런데 그리는 사람마다 도형의 뜻이 다르면, 같은 그림을 보고도 서로 다른 흐름으로 이해하게 됩니다. 그렇다면 누구나 같은 뜻으로 읽는 그림은 어떻게 그릴 수 있을까요?

[SLIDE 27] 문장으로 적은 프로세스

먼저 그림 없이 문장으로 적으면 어떤지 볼게요. 화면 왼쪽이 긴급 대응 프로세스를 문장으로 적은 겁니다. 틀린 내용은 없어요. 그런데 한 번에 읽히시나요?

오른쪽 첫 줄을 보세요. 갈림이 묻혀 있습니다. 승인하면, 정상이면, 아니면, 거절하면. 이런 조건이 문장 여기저기에 흩어져 있죠. 갈래가 몇 개인지 세려면 처음부터 다시 읽어야 해요.

둘째 줄, 책임자가 묻혀 있습니다. 재관측과 보고를 누가 하는지, 문장에는 아예 없어요.

셋째 줄은 더 큰 문제예요. 마지막 문장을 보면, 승인이 10분 넘게 오지 않으면 알림을 보낸다고만 돼 있습니다. 알림을 보낸 다음에도 승인을 계속 기다리는지, 거기서 끝나는지는 알 수가 없죠.

읽는 사람마다 다르게 이해할 수 있다는 겁니다. 사람이 업무를 확인할 때도 문제지만, 이 흐름을 실행할 소프트웨어한테는 훨씬 더 큰 문제예요. 그래서 도형마다 뜻을 약속해 둔 표기가 필요합니다.

[SLIDE 28] BPMN: 업무 프로세스의 표기 표준

그 공통 약속이 BPMN입니다. 업무 프로세스를 표현하는 표준으로, 도형과 선의 의미를 정해 두었습니다. 앞에서 판단 규칙을 다룬 DMN을 정한 OMG가 BPMN도 관리하고, BPMN 2.0은 국제 표준으로도 정해져 있습니다.

공통 표기가 있으면 현업 담당자는 업무의 순서와 책임을 확인하고, 구현하는 쪽은 실행할 흐름을 같은 모델에서 검토할 수 있어요. 문장만으로 놓치던 갈림과 대기를 함께 확인하기 쉬워지는 겁니다.

다만 BPMN으로 그린 모든 그림이 곧바로 실행되는 것은 아닙니다. 업무 설명용 모델도 있고, 실행에 필요한 조건과 연결까지 갖춘 모델도 있어요. 실행 시스템이 지원하는 요소와 데이터, 담당, 외부 호출을 확인해야 합니다. 같은 표준을 쓴다는 이유만으로 어떤 제품에서든 수정 없이 실행된다고 보장할 수는 없어요.

HYD 실습에서는 편집기에서 그린 파일을 가져와 지원 범위를 검사하고, 에이전트와 시스템 연결을 더해 실행 정의를 만듭니다. 우리가 얻으려는 가치는 사람이 합의한 흐름과 실제 실행을 서로 대조할 수 있게 만드는 데 있습니다.

[SLIDE 29] BPMN 요소의 세 묶음

이번 회차에서 다룰 BPMN의 기본 도형은 세 묶음으로 나눠서 읽습니다. 화면 왼쪽부터 흐름 객체, 연결 객체, 스윔레인이에요. 표준에는 데이터나 주석을 나타내는 묶음도 더 있지만, 업무의 흐름을 읽는 데는 이 셋이면 됩니다.

왼쪽 흐름 객체는 프로세스에서 일어나는 일을 나타냅니다. 원은 이벤트, 모서리가 둥근 사각형은 작업, 마름모는 게이트웨이예요.

가운데 연결 객체는 이것들을 잇는 선입니다. 실선 화살표가 있고, 끝이 비어 있는 점선 화살표가 있죠. 이 둘은 뜻이 달라요.

오른쪽 스윔레인은 누가 맡는지를 나누는 칸입니다. 수영장 레인처럼 칸을 나눠서 작업을 배치하는 거예요.

처음 보는 BPMN 그림도 이 세 묶음으로 나눠 보면 훨씬 읽기 쉬워집니다. 무슨 일이 일어나는지, 어떤 순서로 이어지는지, 누가 맡는지. 세 묶음이 이 세 가지를 하나씩 맡고 있거든요. 이번 단원에서는 왼쪽의 흐름 객체와 가운데 실선을, 다음 단원에서는 칸과 점선을 다룰게요.

[SLIDE 30] 긴급 대응 프로세스의 BPMN 그림

이 도형들이 긴급 대응 프로세스에서 어떻게 쓰였는지, 전체 그림부터 보겠습니다. 아까 문장으로 적었던 바로 그 내용을 BPMN으로 그린 거예요. 처음엔 좀 복잡해 보이죠? 왼쪽 칸 이름부터 읽으면 됩니다. 위에서부터 담당자, 에이전트, 시스템이에요.

에이전트 칸 왼쪽에 봉투가 그려진 원, 이게 시작입니다. 쿨러 과열 경보라는 이름이 붙어 있는데, 쿨러 성능 저하 경보가 도착하는 사건을 가리켜요. 거기서 원인 진단과 냉각 조치 제안으로 갑니다.

화살표를 따라가면 위로 올라가서 담당자 칸의 운전원 승인으로 가죠. 그 옆 마름모에는 '승인?'이라는 이름이 붙어 있어요. 승인했는지를 묻는 마름모라서, 앞으로는 승인 여부 게이트웨이라고 부를게요. 여기서 승인과 거절이 갈립니다. 승인 쪽은 시스템 칸으로 내려가서 냉각 명령, 재관측으로 이어지고요. 그다음 다시 마름모에서 유온이 정상인지를 따지는데, 이건 유온 정상 여부 게이트웨이라고 부르겠습니다. 정상이면 작업지시 등록과 정상 보고로 가고, 아니면 미달 보고로 갑니다.

다른 갈래도 있어요. 승인 여부 게이트웨이에서 아래로 내려가는 선, 그리고 운전원 승인 테두리에 붙은 작은 시계에서 나가는 선이요. 이 두 갈래가 어디로 가는지는 도형을 하나씩 배우면서 따라가 보겠습니다. 시계의 뜻은 다음 단원에서 다뤄요.

세어 보면 갈림이 두 곳, 끝이 세 곳이에요. 문장으로는 찾기 어려웠던 갈래가 그림에서는 한눈에 보이죠. 이 그림의 도형을 하나씩 떼어서 보겠습니다.

[SLIDE 31] 이벤트: 프로세스를 여는 사건과 닫는 사건

먼저 원부터요. 원은 이벤트입니다. 프로세스에서 일어나는 사건을 나타내죠. 화면의 원 세 개, 테두리를 견주어 보세요.

왼쪽, 테두리가 가는 원은 시작 이벤트예요. 프로세스를 여는 사건입니다.

가운데는 안에 봉투가 그려진 시작 이벤트예요. 메시지를 받으면 프로세스를 여는, 메시지 시작 이벤트입니다. 긴급 대응 프로세스의 쿨러 과열 경보가 바로 이 모양이죠. 경보 메시지가 도착하면 열리니까요. 앞에서 정한 시작 조건이 그림에서는 이 원 하나로 표현되는 겁니다.

오른쪽, 테두리가 굵은 원은 끝 이벤트예요. 한 갈래를 닫는 사건입니다. 앞에서 완료 조건을 정할 때, 프로세스에 정해 둔 끝 자리에 닿아야 끝난 것이라고 했죠. 흐름의 끝을 나타내는 것이 이 굵은 원, 끝 이벤트입니다. 긴급 대응 프로세스에는 끝, 거절 종결, 알림, 이렇게 세 개가 있어요.

끝 이벤트가 세 개라는 점, 기억해 두세요. 완료 조건 표에서 본 여러 경우가 각자 자기 끝으로 닫히는 겁니다. 이 그림의 일반 끝 이벤트는 도착한 갈래를 마칩니다. 알림 갈래가 끝나도 승인이 남아 있으면 처리 건 전체는 끝나지 않아요. 또 정상과 미달은 같은 끝으로 모이므로, 어떤 끝에 닿았는지에 더해 결과 보고도 읽어야 합니다.

[SLIDE 32] 작업: 사용자 작업과 서비스 작업

다음은 모서리가 둥근 사각형, 작업입니다. 프로세스 안에서 수행되는 일을 활동이라고 하는데, 그 모델 안에서 내부 절차를 더 펼치지 않고 다루는 단위가 작업이에요. 실제로 더 작은 행동이 없다는 뜻은 아닙니다. 상자 세 개의 왼쪽 위 표식을 비교해 보세요.

가운데 운전원 승인에는 사람 표식이 있죠. 사람이 수행하는 사용자 작업입니다. 긴급 대응 그림 전체에서 이 표식이 붙은 작업은 운전원 승인 하나뿐이에요. 그래서 그림만 봐도, 사람이 결정하는 자리가 어디인지 바로 찾을 수 있습니다.

오른쪽 냉각 명령에는 톱니 표식이 있어요. 시스템이 자동으로 수행하는 서비스 작업이고요.

왼쪽 원인 진단과 냉각 조치 제안에는 표식이 없습니다. 누가 수행할지를 그림이 아니라, 실행 전에 따로 지정하는 작업이에요. HYD의 이 예에서는 에이전트에게 맡길 단계를 이 모양으로 그렸습니다.

왜 그럴까요? BPMN 표준에는 에이전트 작업이라는 요소가 따로 없거든요. HYD에서는 그림에 표식 없는 작업을 쓰고, 그 작업을 어느 에이전트가 맡을지는 시스템에 연결할 때 정합니다. 이 연결은 셋째 실습에서 직접 해 볼 거예요.

[SLIDE 33] 시퀀스 흐름과 배타 게이트웨이

이번에는 작업과 작업을 잇는 선, 그리고 흐름이 갈라지는 자리를 볼게요. 화면 왼쪽의 운전원 승인에서 출발하세요.

작업을 잇는 실선 화살표, 이게 시퀀스 흐름입니다. 화살표 방향이 곧 순서예요.

마름모는 게이트웨이입니다. 흐름이 갈라지거나 합쳐지는 지점이죠. 안에 가위표가 그려진 건 배타 게이트웨이라고 해요. 나가는 선 가운데 조건이 참인 하나로만 흐름을 보냅니다.

운전원 승인 뒤의 승인 여부 게이트웨이를 보세요. 앞 장 전체 그림에서 아래로 내려가던 바로 그 갈래가 여기 있어요. 승인이면 위쪽 냉각 명령으로 가고, 거절이면 아래쪽 거절 보고로 갑니다. 둘 중 하나로만 가지, 양쪽으로 동시에 가지는 않아요.

배타라는 말이 바로 그 뜻이에요. 승인과 거절이 함께 일어날 수는 없으니까요.

이 실습에서 갈림을 마름모로 명시하는 이유도 여기에 있어요. 한 작업에서 조건 없는 선을 두 갈래로 뽑으면 두 경로가 함께 진행될 수 있습니다. 표준에는 조건부 시퀀스 흐름도 있으므로 모든 갈림에 마름모가 필수인 것은 아닙니다. 승인과 거절이 동시에 일어나는 그림이 되어 버리는 거죠. 그래서 실습에서 그린 그림을 HYD에 가져올 때도, 마름모 없이 갈라진 흐름은 등록을 받지 않습니다.

[SLIDE 34] 갈림의 조건과 기본 흐름

그렇다면 어느 쪽으로 갈지는 뭘로 정할까요? 게이트웨이에서 나가는 선에는 조건이 붙습니다. 표의 오른쪽 열이 그 조건이에요.

첫 줄을 보세요. 승인 여부 게이트웨이에서 냉각 명령으로 가는 선은, 운전원 승인 작업이 낸 승인 결과가 승인일 때 참이 됩니다.

셋째 줄은 유온 정상 여부 게이트웨이예요. 작업지시 등록으로 가는 선은 재관측이 낸 회복 여부가 참일 때 갑니다.

둘째 줄과 넷째 줄에는 조건 대신 기본 흐름이라고 적혀 있죠. 어느 조건도 참이 아닐 때 가는 선이에요. 그림에서는 선이 출발하는 자리에 짧은 빗금을 그어서 표시합니다. 앞 장 그림에서 거절 쪽 선이 출발하는 자리에 있던 그 빗금이에요.

여기서 중요한 건, 조건이 앞 작업이 낸 값을 견주는 식이라는 점이에요. 갈림을 그때그때 누군가 해석해서 정하지 않습니다. 조건을 문장으로 적어 두고 모델에게 해석을 맡기지도 않고요. 같은 정의와 조건식에 같은 값을 넣으면 같은 갈래로 가고, 나중에 왜 그 길로 갔는지도 기록으로 설명할 수 있습니다.

기본 흐름을 두는 까닭도 여기 있어요. 기본 흐름이 없는데 어느 조건도 맞지 않으면, 흐름은 그 자리에서 넘어가지 못하고 멈춥니다. 그러면 처리를 이끄는 시스템은 그 자리에, 왜 넘어가지 못했는지 사유를 남겨요.

판단은 에이전트가 하지만, 갈림은 이렇게 정해진 조건이 합니다. 문장 속에 묻혀 있던 순서와 갈림이, 도형과 조건으로 드러난 거죠.

조건이 여러 개라면 동시에 참이 될 수 있는지도 살펴야 합니다. 배타 분기에서는 어느 하나를 택할 규칙이 분명해야 해요. 기본 흐름도 누락된 입력을 무조건 정상 결과로 처리하는 장치가 아닙니다. 필요한 값은 먼저 검증하고, 어느 조건에도 해당하지 않는 유효한 경우를 어디로 보낼지 정합니다.

[SLIDE 35] BPMN: 레인 · 메시지 흐름 · 타이머

순서와 갈림은 이렇게 그림에 담겼는데요, 아직 나타나지 않은 게 있습니다. 다섯 번째 단원에서는 그걸 그려 넣어요. 각 작업은 누가 맡을까요? 그리고 운전원이 응답하지 않으면 어떻게 될까요? 작업마다 맡는 쪽을 나누는 칸과, 기다리는 동안 생기는 일을 그리는 도형을 볼 거예요. 그러면서 프로세스를 정의할 때 정한 책임자와 완료 조건이, 그림의 어디에 들어가는지 찾아보겠습니다.

[SLIDE 36] 풀과 레인: 참여자와 책임의 칸

작업을 맡는 쪽을 그림에서 어떻게 읽을까요? 바깥 테두리인 풀과, 그 안을 나눈 레인을 구분해 보세요.

풀은 참여자를 나타내는 영역입니다. 조직이나 시스템을 참여자로 표현할 수 있어요. 레인은 풀 안의 활동을 묶어 구분하는 칸이고, 여기서는 담당 역할을 나타내는 데 썼습니다. 원인 진단은 에이전트 레인, 승인은 담당자 레인, 냉각 명령은 시스템 레인에 놓였죠.

레인의 분류 기준은 모델을 만드는 사람이 정합니다. 언제나 역할이어야 하는 것은 아니에요. 또 그림에서 상자를 옮겼다는 사실만으로 실제 계정 권한이나 담당 시스템이 바뀌는 것도 아닙니다. 이 수업에서는 레인에 표시한 수행 주체와 실행 연결을 맞춰 봅니다.

HYD는 칸 이름이 역할 이름과 일치하면 연결을 제안하고 사람이 바꿀 수 있게 합니다. 레인과 단계의 담당이 어긋나는지도 검사하죠. 그림이 나타내는 책임과 실제 배정이 일치하는지를 확인하는 과정입니다.

같은 풀 안에서는 레인이 달라도 시퀀스 흐름으로 이을 수 있습니다. 서로 다른 풀 사이의 주고받음은 다음에 볼 메시지 흐름으로 구별합니다.

[SLIDE 37] 사람, 에이전트, 시스템의 책임 배치

칸이 곧 맡는 쪽이라면, 긴급 대응 프로세스에서는 실제로 누가 맡을까요? 쿨러 사건처럼 승인된 경우를 표로 정리했습니다. 가운데 열이 맡는 쪽, 오른쪽 열이 그 작업이 놓인 레인이에요.

첫 줄, 원인 진단과 냉각 조치 제안은 냉각 긴급 대응 에이전트가 맡고, 에이전트 레인에 놓입니다.

둘째 줄, 운전원 승인은 운전원 역할이 맡고 담당자 레인에 놓이고요.

나머지 네 줄은 전부 시스템 레인이에요. 냉각 명령은 SCADA, 작업지시 등록은 CMMS가 맡고, 재관측과 결과 보고는 처리를 이끄는 시스템이 직접 맡습니다.

표를 위에서 아래로 다시 따라가 보세요. 판단은 에이전트 레인에, 결정은 담당자 레인에, 실행과 기록은 시스템 레인에 있습니다. 그리고 설비를 움직이는 냉각 명령 앞에는 반드시 사람의 승인이 놓여 있죠. 1회차에서 추천과 실행의 책임 경계를 가려 봤었는데, 그 경계가 여기서는 그림의 칸으로 그려진 겁니다. 누가 판단하고 결정하고 실행하는지를 그림에 나타내고, 실제 연결과 권한이 그 배치를 따르는지 확인하는 겁니다.

[SLIDE 38] 메시지 흐름: 풀과 풀 사이의 주고받음

지금까지는 풀 하나 안에서 칸을 나눴어요. 그럼 참여자가 둘이면 어떻게 그릴까요? 화면은 설비를 운영하는 조직이 부품 공급사에 발주하는 일반적인 예예요. 위아래로 풀이 두 개 있죠.

각 풀 안에서 작업 순서를 잇는 선은, 앞에서 본 실선의 시퀀스 흐름입니다.

그런데 풀과 풀 사이를 오가는 선은 점선이죠. 이게 메시지 흐름이에요. 출발 쪽에 작은 원이 있고, 화살촉이 비어 있습니다. 운영 조직이 발주서를 보내고, 공급사가 납기 회신을 보내는 주고받음을 나타내요.

두 선을 왜 구분할까요? 시퀀스 흐름은 한 참여자 안에서 정해진 순서예요. 메시지 흐름은 서로 다른 참여자끼리 주고받는 거고요. 공급사는 자기 절차로 주문을 처리하고, 운영 조직은 회신을 받아 다음 작업을 진행합니다. 메시지 흐름은 이 참여자 사이의 통신을 나타내며, 그 선만 그었다고 실제 전송 연결까지 생기는 것은 아니에요.

긴급 대응 프로세스는 풀 하나에 레인 셋으로 그리니까 메시지 흐름이 없습니다. 실습에서도 풀 하나로 그려요. 풀을 둘로 나눠 그린 그림을 가져오면, 한 풀 안에 레인으로 나눠 그리라는 안내가 나오면서 받아들여지지 않습니다.

[SLIDE 39] 경계 이벤트와 타이머

이제 남은 물음으로 넘어갈게요. 운전원이 응답하지 않으면 어떻게 될까요? 작업이 끝나기를 기다리는 사이에 무슨 일이 일어나면, 작업 테두리에 붙인 경계 이벤트로 그립니다.

화면의 두 그림을 보면, 둘 다 작업 아래 테두리에 겹테두리 원이 붙어 있어요. 원 안에 시계가 그려져 있죠. 정한 기간이 지나면 발생하는 타이머입니다.

테두리에 붙인다는 게 중요해요. 이 타이머는 붙은 작업이 진행 중일 때만 살아 있습니다. 작업이 기한 안에 끝나면, 테두리의 타이머도 함께 거둬져요. 운전원이 제때 승인하면 지연 알림은 나가지 않는 거죠. 기한이 지났을 때 무슨 일이 생기는지는 두 그림이 서로 다릅니다.

왼쪽은 실선 겹테두리, 중단형이에요. 타이머가 발생하면 붙어 있던 작업을 끊고 다른 가지로 갑니다. 그래서 위의 작업이 흐려져 있고, 가위표가 붙어 있죠.

오른쪽은 점선 겹테두리, 비중단형입니다. 타이머가 발생해도 작업은 그대로 두고, 가지를 하나 더 열어요. 그래서 위의 작업이 진한 채로 남아 있습니다.

선이 실선이냐 점선이냐, 이것 하나로 결과가 완전히 달라집니다. 끊느냐, 그대로 두느냐죠. 문장으로 적었을 때 알 수 없었던 물음, 기억나세요? 알림 뒤에도 기다리느냐. 그게 바로 이 구분으로 그림에 드러나는 겁니다.

BPMN에는 중간 타이머 같은 다른 표현도 있지만, 현재 HYD 가져오기에서 지원하는 기다림은 작업의 경계 타이머입니다. 흐름 중간에 타이머를 따로 세워 둔 그림은 가져올 때 받지 않으니, 실습에서도 타이머는 작업 테두리에 붙여 그려요.

[SLIDE 40] 승인을 받는 자리의 그림

긴급 대응 프로세스에서 사람의 결정을 받는 부분만 떼어 놓았어요. 가운데 운전원 승인이 중심입니다.

운전원 승인 바로 뒤에 승인 여부 게이트웨이가 있죠. 승인이면 오른쪽 냉각 명령으로 가고, 거절이면 거절 보고를 거쳐서 거절 종결로 닫힙니다.

그리고 운전원 승인의 테두리를 보세요. 전체 그림에서 뜻을 미뤄 두었던 그 작은 시계가, 앞 장에서 본 경계 타이머예요. 이름은 승인 지연이고, 점선, 그러니까 비중단형이고, 기한은 10분입니다. 라벨에는 PT10M이라고 적혀 있죠. 기간을 적는 국제 표준 표기로, P는 기간, T 뒤의 10M은 10분이라는 뜻이에요. 셋째 실습에서 타이머의 기한을 넣을 때 이 꼴로 적습니다. 이 실습의 20배속에서는 10분이 실제 30초로 적용됩니다. 그 기한이 지나면 왼쪽 아래의 승인 지연 알림 가지만 열리고, 승인 작업은 계속 기다려요. 완료 조건에서 본 응답 없음의 경우가 그림에서는 이렇게 표현되는 겁니다.

만약 이 타이머를 실선으로 그렸다면 어떻게 될까요? 10분이 지나는 순간 승인 작업이 끊기고, 운전원은 더 이상 승인할 수 없게 되죠. 사람의 결정을 시간에 밀려 잃지 않도록, 비중단형으로 그린 겁니다.

그렇다고 실선이 틀린 건 아니에요. 실선으로 그리는 설계도 실제로 있습니다. HYD에 원래 들어 있는 기준 흐름이 그래요. 운전원이 조치를 고르지 않고 10분이 지나면, 그 작업을 끊고 상급자를 부르는 가지로 넘어갑니다. 응답이 늦을 때 결정을 다른 사람에게 넘길지, 알림만 보내고 같은 사람을 기다릴지는 업무를 설계하는 사람이 정해요. 그 선택이 그림에서는 선 하나의 모양으로 드러나는 거죠.

[SLIDE 41] 같은 그림 위의 세 가지 경로

이렇게 그림 하나가 여러 경우를 담고 있어요. 화면의 카드 세 장은 같은 그림 위에서 갈 수 있는 세 경로입니다. 카드마다 붙은 꼬리표가 그 경로가 닿는 끝이고요.

첫 번째 카드는 승인 뒤 회복도 확인된 경로입니다. 쿨러 사건이 이 길로 갔죠. 냉각 명령, 재관측, 유온 정상 여부 게이트웨이, 작업지시 등록, 정상 보고를 거쳐서 끝에 닿습니다.

두 번째 카드, 거절이에요. 승인 여부 게이트웨이에서 거절 보고로 가고, 거절 종결에 닿습니다. 냉각 명령으로 가는 선은 지나지 않아요.

세 번째 카드, 응답 없음입니다. 승인 지연 타이머가 알림 가지를 열고, 그 가지만 알림에 닿아서 끝나요. 운전원 승인은 열린 채로 남고요.

꼬리표 셋을 다시 보세요. 끝, 거절 종결, 알림. 이벤트를 배울 때 본 끝 이벤트 셋이 이 세 경로와 하나씩 짝을 이룹니다. 다만 세 번째 경로는 다른 둘과 달라요. 알림에 닿아도 승인 작업이 아직 열려 있으니, 처리 건 전체는 끝나지 않습니다. 경우마다 그림을 따로 그리지 않고, 그림 하나에 갈림과 끝을 모두 담아 둔 덕분에, 각 처리 건이 어떤 경로를 지났는지 설명할 수 있어요. 알림과 승인 대기처럼 동시에 열린 갈래가 있을 수도 있습니다.

[SLIDE 42] 병렬 게이트웨이와 하위 프로세스

마지막으로, 프로세스가 커지면 쓰게 되는 요소 두 개를 개념만 소개할게요.

화면 왼쪽, 더하기표가 그려진 마름모가 병렬 게이트웨이입니다. 배타 게이트웨이는 한 갈래만 골랐죠? 병렬 게이트웨이는 나가는 모든 갈래로 흐름을 함께 보내요. 그리고 뒤쪽의 합류 마름모에서는, 모든 갈래가 끝날 때까지 기다렸다가 다음으로 갑니다.

화면 오른쪽은 하위 프로세스예요. 여러 작업으로 된 흐름을 작업 하나처럼 접어서 그린 겁니다. 위쪽이 접힌 모양인데, 아래 가운데의 더하기 표식이 안에 흐름이 들어 있다는 뜻이에요. 아래쪽이 그걸 펼친 모양이고요.

긴급 대응 프로세스에는 둘 다 쓰이지 않았습니다. 이번 회차 실습에서도 하위 프로세스로 접지 않고, 한 흐름으로 펼쳐서 그려요. 이렇게 해서 누가 맡는지는 레인으로, 응답이 늦을 때의 일은 타이머로 그려 넣었고, 긴급 대응 프로세스의 그림이 완성됐습니다.

다음 시간에는 이 그림을 실제로 실행하는 쪽, 그리고 실행 중인 처리 건이 지금 어디까지 왔는지를 읽는 법을 보겠습니다.

[이론 03] 설계한 업무가 실제로 실행되는 프로세스 오케스트레이션 이해하기

[SLIDE 43] 프로세스 엔진과 처리 건의 진행

이번 시간에는 그린 프로세스가 실제로 실행되는 방식, 프로세스 오케스트레이션을 봅니다. 앞 시간에는 긴급 대응 프로세스의 BPMN 그림을 읽었습니다. 직접 작도하는 실습은 이론을 마친 뒤에 이어집니다. 그런데 그 그림은 그 자체로는 설계도예요. 경보가 왔을 때 그림대로 한 단계씩 일이 일어나려면, 그림을 읽고 움직이는 쪽이 있어야겠죠. 그 실행 주체는 무엇이고, 실행 중인 처리 건이 지금 어디까지 왔는지는 어떻게 알 수 있을까요?

[SLIDE 44] 그림만으로는 일어나지 않는 일

실행 주체가 맡아야 할 일부터 정리해 볼게요. 화면의 카드 세 장, 세 가지입니다.

첫 번째 카드, 다음 작업을 열어야 합니다. 에이전트가 추천안을 제출하면, 누군가는 운전원 앞에 승인 작업을 놓아야 하죠.

두 번째 카드, 결과를 넘겨야 해요. 운전원이 승인했다는 결과를 승인 여부 게이트웨이에 전해야 하고, 재관측의 판정도 다음 갈림에 전해야 합니다.

세 번째 카드, 위치를 기억해야 합니다. 운전원이 한참 뒤에 응답하더라도, 이 처리 건이 어디까지 왔는지를 잊지 않고 있어야 해요.

이 세 가지를 사람이 챙긴다면 어떻게 될까요? 앞에서 본 AS-IS로 돌아가는 셈이죠. 그래서 그림을 읽고 이 일을 대신 해 줄 실행 주체가 필요합니다.

[SLIDE 45] 프로세스 엔진과 오케스트레이션

그 실행 주체가 프로세스 엔진이에요. 프로세스의 정의를 읽어서 그대로 실행하는 소프트웨어입니다. HYD에서는 경보로 열린 처리 건을 이 엔진이 정의의 조건에 따라 진행을 관리합니다.

앞에서 계속 처리를 이끄는 시스템이라고 불렀던 게 바로 이 엔진입니다. 재관측과 결과 보고를 직접 맡던 그쪽이죠.

둘째 줄은 오케스트레이션이에요. 중앙의 조정자 하나가 작업의 순서와 호출을 이끄는 방식입니다. 사람, 에이전트, 시스템은 각자 자기 작업만 하고, 다음이 누구 차례인지는 엔진이 정해요. 이렇게 한 곳이 순서를 쥐고 있으면, 처리 건이 지금 어디까지 왔는지와 누가 무엇을 했는지를 엔진 한 곳에서 볼 수 있습니다. 사람과 에이전트와 시스템에게 하나하나 물어보러 다닐 필요가 없죠.

셋째 줄은 비유입니다. 오케스트라에서 연주자는 자기 악기만 맡고, 언제 들어올지는 지휘자가 알려 주죠. 여기까지가 닮은 점이고요, 다른 점도 있어요. 지휘자는 차례만 알려 주지만, 엔진은 작업의 결과를 받아서 다음 작업에 넘기고, 그 과정을 기록까지 하거든요.

여기서 중앙이라는 말은 조정 책임이 한곳에 있다는 뜻입니다. 반드시 컴퓨터 한 대에서 모두 실행된다는 뜻은 아니에요. 연결된 시스템이 실패하면 엔진도 그 결과를 받아 멈추거나 정해 둔 대응 경로로 보내야 합니다. 어떤 엔진을 고르더라도 진행 상태와 실패 이유, 다시 처리할 기준이 남는지를 확인해야 해요.

[SLIDE 46] 프로세스 정의와 프로세스 인스턴스

엔진이 다루는 걸 둘로 나눠서 봐야 합니다. 화면 왼쪽의 설계도 한 장, 그리고 오른쪽에 실행되는 여러 건이에요.

왼쪽은 프로세스 정의입니다. 요소와 선과 조건을 담은 설계도예요. 긴급 대응 프로세스의 설계도는 이렇게 한 장입니다.

오른쪽은 프로세스 인스턴스예요. 그 설계도에 따라 실제로 실행되는 한 건이죠. 앞에서 처리 건이라고 부른 게 바로 이겁니다. 경보가 올 때마다 하나씩 만들어져요.

오른쪽 세 줄을 보세요. 줄마다 그림의 모양이 다르죠. 맨 위가 쿨러 사건의 처리 건인데, 처음부터 끝까지 다 지나간 모양이에요. 승인돼서 끝까지 갔습니다. 아래 두 줄은 같은 정의에서 열린 다른 처리 건이에요. 가운데 줄은 중간에서 끝난 모양으로, 거절로 끝난 건입니다. 맨 아래 줄은 도중에 머물러 있는 모양이죠. 이 줄은 그 건이 승인을 기다리던 순간을 그린 거예요.

같은 설계도에서 나왔어도, 인스턴스마다 이렇게 다르게 진행되는 거죠.

[SLIDE 47] 토큰: 프로세스의 현재 위치

인스턴스마다 진행이 다르다면, 한창 진행 중인 인스턴스가 지금 어디에 있는지는 어떻게 나타낼까요? 그 위치를 설명하는 개념이 토큰이에요. 프로세스의 현재 위치를 나타내는 표식입니다. 그림에서 운전원 승인 오른쪽 위에 있는 파란 점, 보이시죠?

토큰은 시작 이벤트에서 하나 생깁니다. 그리고 작업이 끝날 때마다 나가는 선을 따라 다음 요소로 옮겨 가요. 게이트웨이에서는 조건이 참인 선을 따라가고, 끝 이벤트에 닿으면 사라집니다.

지금 그림은 토큰이 운전원 승인에 있는 순간이에요. 에이전트의 작업은 이미 끝났고, 냉각 명령에는 아직 닿지 않았다는 뜻이죠. 그러니까 이 처리 건은 지금 운전원 차례입니다.

이 점이 처음부터 끝까지 움직인 모습을 쿨러 사건으로 따라가 볼까요? 토큰은 경보가 온 순간 생겨서, 에이전트 단계에 꽤 오래 머물렀어요. 운전원 승인에서는 짧게 머물렀고, 냉각 명령은 순식간에 지나갔죠. 그다음 재관측에서 다시 한동안 머문 뒤, 작업지시 등록과 정상 보고를 지나 끝에서 사라졌습니다. 처리 건의 기록에는 단계마다 흐름이 닿은 시각과 끝난 시각이 남는데, 그게 토큰이 지나간 흔적이에요. 그러니까 토큰이 어디에 오래 머물렀는지를 보면, 처리 건의 시간이 어디

에 쓰였는지가 보입니다. BPM의 모니터링에서 단계마다의 대기를 지켜본다고 했던 게, 바로 이걸 읽는 일이에요.

앞에서 본 병렬 게이트웨이도 토큰으로 설명할 수 있어요. 병렬 게이트웨이를 지나면 토큰이 갈래 수만큼 나뉘고, 합류 자리에서는 모든 토큰이 도착할 때까지 기다렸다가 하나로 합쳐집니다.

토큰은 BPMN이 실행을 설명할 때 쓰는 표준 개념이에요. HYD 화면에서는 토큰이라는 물체 대신 지금 열려 있는 단계와 기록으로 진행을 확인합니다. 언어 모델의 입력 길이를 세는 토큰과는 다른 개념이에요.

[SLIDE 48] 승인 대기: 토큰이 사용자 작업에 머문 상태

열려 있는 단계가 실제 기록에서는 어떻게 보이는지, 운전원의 승인을 기다리던 처리 건으로 확인해 볼게요. 쿨러 사건과 같은 정의에서 열렸지만, 운전원이 한동안 응답하지 않았던 다른 처리 건입니다.

화면 맨 위 줄은 처리 건의 상태예요. 실행 중이라는 뜻입니다. 아직 끝나지 않았으니까요.

둘째 줄은 지금 열려 있는 단계, 운전원 승인이죠. 토큰이 있는 곳이 바로 여기예요.

그 아래는 단계마다의 상태입니다. 여기서 단계라는 말을 짚어 둘게요. 이 흐름에서는 그림의 작업을 처리 건의 단계로 기록합니다. 그림의 운전원 승인은 설계도에 그린 작업이고, 이 처리 건의 운전원 승인 단계는 그 작업이 이 건에서 실제로 실행되는 자리예요. 마지막 실습에서 처리 건 화면을 볼 때도 이 단계라는 말로 읽게 됩니다.

이 건의 단계를 보면, 에이전트 단계는 끝났고, 운전원 승인은 진행 중이고, 냉각 명령부터 그 뒤로는 아직 닿지 않았어요. 앞 장 그림에서 토큰이 있던 자리와 딱 맞죠.

하나 더 보세요. 아직 닿지 않은 단계가 처음부터 기록에 있습니다. 처리 건이 열릴 때, 앞으로 닿을 수 있는 단계를 미리 만들어 두거든요. 그래서 지금 어디에 있는지뿐 아니라, 앞으로 무엇이 남았는지도 처리 건 하나에서 볼 수 있어요.

그리고 꼭 기억해 둘 게 있습니다. 승인 대기라는 이유만으로 오류라고 읽으면 안 돼요. 다만 담당자가 없거나 알림 전달이 실패해 대기가 길어졌는지는 별도로 살펴야 합니다. 토큰이 사용자 작업에 머물러서 사람의 결정을 기다리는, 정상적인 실행 상태입니다. 승인 대기와 정상 종료를 구분하는 건 마지막 실습에서 직접 확인할 내용이기도 해요.

[SLIDE 49] 토큰이 둘이 되는 순간

같은 처리 건에서, 기한이 지나도록 운전원이 응답하지 않았을 때예요. 이번에는 파란 점이 두 개죠.

승인 작업에 붙은 승인 지연 타이머가 발생했습니다. 비중단형이니까 작업을 끊지 않고 가지를 하나 더 열었죠. 그러면서 토큰이 하나 더 생겼어요. 왼쪽 아래를 보시면, 둘째 토큰이 알림 가지를 지나 승인 지연 알림에 있습니다.

처음의 토큰은 어디 있을까요? 운전원 승인에 그대로 있어요. 그래서 알림이 나간 뒤에도 처리 건은 닫히지 않습니다. 이 그림은 알림 가지가 진행되는 순간을 설명합니다. 저장된 대기 스냅샷에서는 알림 단계가 이미 끝났고 승인 단계만 진행 중이었어요. 알림이 생겼던 사실과 지금 열려 있는 단계 수를 구분해야 합니다.

운전원은 여전히 승인하거나 거절할 수 있습니다. 실제로 이 처리 건은 그 뒤에 거절로 닫혔고요. 그림의 점선 하나가, 실행에서는 토큰 하나를 더 만드는 걸로 나타난 거예요. 이렇게 프로세스 엔진이 그림을 따라 토큰을 옮기기 때문에, 처리 건이 지금 어디까지 왔는지를 언제든 읽을 수 있습니다.

[SLIDE 50] 프로세스 변수와 에이전트 단계

일곱 번째 단원에서는 토큰이 지나가는 작업과 작업 사이에, 무엇이 오가는지 봅니다. 승인 여부 게이트웨이가 승인인지 거절인지를 가리려면, 앞 작업이 낸 결과를 받아야 하잖아요. 작업의 결과는 다음 작업으로 어떻게 전해질까요? 그리고 에이전트의 판단은 그 흐름 속 어디에 놓일까요? 결과를 담아 나르는 것부터 보겠습니다.

[SLIDE 51] 프로세스 변수(Process Variable)

작업의 결과가 다음으로 전해지는 길, 그게 프로세스 변수입니다. 인스턴스가 지니고 다니는 이름 붙은 값이에요.

표를 보시면 왼쪽이 변수 이름, 가운데가 그 값을 만드는 단계, 오른쪽은 쿨러 사건의 결과를 변수의 의미에 대응시킨 예입니다. 그 처리 건의 변수 원문을 그대로 옮긴 표는 아니에요.

첫 줄, 원인은 에이전트 단계가 만들어요. 쿨러 사건에서는 쿨러 핀 오염이었죠.

둘째 줄을 보세요. 승인 결과는 운전원 승인이 만들고, 승인 여부 게이트웨이가 이 값을 읽습니다. 앞 단원에서 본 갈림의 조건, 그 조건이 읽던 값이 바로 이 변수예요.

셋째 줄은 운전원이 승인한 조치, 팬 최대 운전이고요. 넷째 줄의 회복 여부는 재관측이 만들고, 유온 정상 여부 게이트웨이가 읽습니다. 마지막 줄은 작업지시 등록이 만든 작업지시 번호예요. 결과 보고의 문장도 이 변수로 채워집니다. 정상 보고에서 작업지시 번호가 들어갈 자리에 이 값이 들어가죠. 프로세스를 정의할 때 본 출력과 입력의 연쇄가, 실행에서는 이렇게 변수로 이어지는 겁니다.

처리 건은 변수마다 그걸 만든 단계를 같이 적어 둡니다. 그래서 어떤 값이 이상하다 싶으면, 그 값이 어느 단계에서 나왔는지 거슬러 올라갈 수 있어요.

그리고 누가 승인했는지, 어느 역할로 승인했는지 같은 값은 승인하는 순간 시스템이 직접 채웁니다. 에이전트가 이 값을 대신 써 넣을 수는 없어요. 책임에 관한 값은, 책임을 진 쪽의 행동으로만 생기게 해 둔 거죠.

[SLIDE 52] 작업의 종류에 따라 엔진이 기다리는 것

토큰이 작업에 닿으면 엔진은 뭘 할까요? 작업의 종류에 따라 다르게 움직이고, 기다리는 것도 다릅니다. 화면의 카드 세 장을 볼게요.

첫 번째 카드, 사용자 작업이에요. 운전원 승인처럼 사람이 하는 작업이면, 엔진은 그 작업을 담당 역할의 작업함에 놓습니다. 그리고 사람이 승인하거나 거절할 때까지 기다려요.

두 번째 카드, 서비스 작업입니다. 냉각 명령, 재관측, 작업지시 등록이 여기 해당하죠. 엔진이 해당 시스템을 직접 호출하고, 결과를 받아 완료 조건을 확인합니다. 호출 실패나 필요한 결과의 누락이 있으면 성공한 것으로 넘겨서는 안 됩니다.

세 번째 카드, 에이전트 단계예요. 원인 진단과 냉각 조치 제안처럼 에이전트가 맡은 작업이면, 엔진은 지정된 에이전트에게 일을 맡기고 추천안이 제출되기를 기다립니다. 여기서 중요한 점이 있어요. 에이전트가 다음 작업을 직접 열지는 않습니다. 정해진 결과를 제출하고 검사가 끝나면, 그 다음을 여는 건 엔진이에요. 제출했다는 사건과 다음 단계로 진행됐다는 사건을 구분해 기록합니다.

어느 경우든 작업이 끝나야 엔진이 토큰을 다음으로 넘깁니다. 사람의 승인도 그냥 넘어가지 않아요. 앞에서 본 역할 확인을 거친 뒤에도, 엔진은 승인 내용이 다음 단계로 제대로 전달된 뒤에야 냉각 명령을 엽니다. 전달에 실패하면요? 다음으로 넘어가지 않습니다.

[SLIDE 53] 에이전트 단계 안에서 읽는 것

에이전트 단계 안을 한번 들여다볼게요. 화면에는 냉각 긴급 대응 에이전트가 따르는 스킬이 정한 순서를 왼쪽부터 놓았습니다.

첫 칸, 먼저 작업의 입력을 읽어요. 어느 설비의 어떤 경보인지 확인하는 거죠.

둘째 칸, 원인 진단 도구로 지식 그래프에서 원인 후보와 증거를 따라가고, 센서 값을 읽어서 원인을 가립니다. 온톨로지가 쓰이는 자리가 바로 여기예요.

셋째 칸에는 필요하면이라고 적혀 있죠. 필요할 때 업무 DB에서 납기가 걸린 생산 오더와 출하를 기다리는 품질 로트를 확인합니다. 설비 값만 보면 될 것 같은데, 왜 업무 값까지 볼까요? 조치 후보 가운데에는 부하를 낮추는 것처럼 생산에 영향을 주는 대안이 있거든요. 납기가 걸린 주문이 있는지, 출하를 기다리는 제품이 있는지에 따라 같은 조치라도 손익이 달라집니다. 물론 여기서도 읽기만 하고, 바꾸지는 않습니다.

넷째 칸, 대안 평가 도구로 SOP의 조치 후보 네 개를 판단 규칙과 성과 기준으로 견주어서 순위를 냅니다. 납기나 품질 같은 업무 값이 순위에 들어가는 건 바로 이 도구가 점수를 셀 때예요. 셋째 칸은 에이전트가 그 값을 직접 확인해 보는 자리이고, 순위 계산은 이 평가 도구가 맡습니다.

마지막 칸, 추천안 하나를 제출해요. 제출하면 판단 기록 번호가 붙습니다.

쿨러 사건에서도 에이전트는 이 한 단계 안에서 도구를 여러 차례 불렀어요. 이 단계에서 진단과 대안 평가에 필요한 자료를 도구로 확인한 겁니다. 화면은 스킬이 안내하는 순서이며 모든 실행의 호출 순서가 같다는 뜻은 아니에요. 그렇게 나온 추천안에는 팬 최대 운전이 1위로 담겼고, 공동 2순위 둘과 쿨러 핀 세척도 지는 대안으로 함께 담겼어요. 왜 추천 하나만이 아니라 대안과 그 까닭까지 담을까요? 승인하는 사람이, 무엇을 두고 고르는 건지 알 수 있게 하려는 거예요.

[SLIDE 54] 프로세스가 에이전트의 결과를 잇는 순서

이 한 단계의 결과가 단계 밖으로는 어떻게 이어졌을까요? 쿨러 사건의 처리 건이 실제로 지나간 순서로 보겠습니다. 이번에는 오른쪽 열, 각 작업이 실제로 낸 값을 따라가 볼게요.

첫 줄, 에이전트 단계가 낸 건 원인과 추천입니다. 쿨러 핀 오염, 그리고 팬 최대 운전이죠. 이 첫 줄에서 진단과 추천을 위한 자료를 읽었어요. 뒤의 승인 검사와 재관측에서는 실행 시점의 자료를 다시 확인합니다.

둘째 줄부터는 프로세스가 이어 갑니다. 운전원은 추천안을 승인했고, 승인된 조치는 냉각 명령으로 가서 팬 속도를 최대로 올렸어요. 안전 연동 검사도 통과했고요.

재관측에서는 유온을 다시 읽었습니다. 60도를 넘던 유온이 50도 남짓까지 내려와서, 회복 기준인 55도 아래로 들어왔고 경보 해제도 확인됐죠. 회복이 확인되니까 작업지시가 등록됐고, 마지막으로 이상이 완화되고 경보가 해제됐다는 정상 보고가 나갔습니다.

에이전트가 무엇을 읽을지는 에이전트가 정해요. 하지만 그 결과가 어떤 순서로 누구에게 가는지는 프로세스가 정합니다. 에이전트의 판단은 흐름 속 한 단계에 놓이고, 그 결과는 변수에 실려 다음 작업으로 넘어가요. 온톨로지와 DB와 SOP를 쓰는 에이전트의 판단이, 이렇게 승인과 명령과 재관측으로 이어져서 끝난 업무가 되는 겁니다.

[SLIDE 55] 실행 상태 판본 · 감사 기록

마지막 이론 단원에서는 끝난 처리 건을 읽습니다. 쿨러 사건의 처리 건은 정상 보고로 끝났죠. 이렇게 끝난 처리 건에서는 뭘 확인할 수 있어야 할까요? 그 결말을 어떻게 판정하는지, 흐름을 고치면 언제부터 바뀌는지, 그리고 누가 무엇을 결정했는지입니다. 처리 건의 상태와 판본과 기록을 차례로 읽어 보겠습니다.

[SLIDE 56] 실행 상태: 처리 건의 상태와 단계의 상태

실행 상태는 두 층으로 읽어요. 화면 위 두 줄은 처리 건 전체의 상태, 아래 두 줄은 단계 하나하나의 상태입니다.

여기서는 이 사례에서 비교할 처리 건의 상태 두 가지를 골랐습니다. 전체 상태 목록을 보여 주는 표는 아니에요. 화면의 RUNNING은 진행 중이라는 상태, COMPLETED는 끝 이벤트에 닿아서 실행이 끝났다는 상태, 줄여서 끝남이죠. 승인을 기다리는 동안에도 처리 건은 진행 중입니다.

단계의 상태는 열려 있다, 끝났다, 아직 닿지 않았다, 이 셋이 기본이에요. 승인 대기 기록에서 본 그대로입니다.

그런데 마지막 줄에 하나가 더 있죠. 화면의 CANCELLED, 취소됨이라는 상태인데요, 이 예에서는 게이트웨이에서 선택되지 않은 갈래의 단계에 붙었습니다. 다른 상황의 취소 사유까지 이 뜻 하나로 읽지는 않습니다. 거절된 처리 건에서는 냉각 명령이 이 상태로 남아요.

그러니까 이 두 처리 건 상태만으로 업무 결과까지 알 수는 없습니다. 어디에서 기다리고 있는지, 어느 갈래로 갔는지는 단계의 상태를 봐야 알 수 있어요.

[SLIDE 57] COMPLETED는 성공을 뜻하지 않는다

이 구분이 왜 중요할까요? 같은 정의에서 나온 처리 건 두 개를 나란히 놓고 보겠습니다. 왼쪽은 승인된 처리 건, 오른쪽은 거절된 처리 건이에요. 두 칸의 맨 위 줄을 견주어 보세요.

둘 다 상태가 똑같이 COMPLETED, 끝남이죠.

그런데 내용은 전혀 다릅니다. 왼쪽은 끝이라는 끝 이벤트에 닿았고, 결과 보고는 정상, 설비 명령이 한 건 나갔어요. 오른쪽은 거절 종결에 닿았고, 결과 보고는 거절, 설비 명령은 없습니다. 기록에는 거절이 반려라는 값으로 적혀 있고요. 냉각 명령, 재관측, 작업지시 등록은 모두 취소됨으로 남았습니다.

그러니까 COMPLETED는 실행이 끝났다는 뜻일 뿐이에요. 업무가 잘됐다는 뜻이 아닙니다. 업무가 어떻게 끝났는지는 닿은 끝 이벤트와 결과 보고로 읽어야 해요.

왼쪽 칸 마지막 줄도 보세요. 승인했는데 회복이 미달인 경우도, 끝 이벤트는 정상일 때와 같은 끝입니다. 그때는 결과 보고가 정상인지 미달인지를 봐야 구분할 수 있어요.

[SLIDE 58] 판본과 배포: 고친 프로세스가 적용되는 시점

BPM의 개선 단계에서 프로세스를 고쳤다고 해 볼게요. 고친 내용은 언제부터 적용될까요? 화면 왼쪽부터 네 칸을 따라가 보겠습니다.

첫 칸, 고친 정의를 새 판본으로 등록합니다. 한번 저장된 판본은 고치지 않아요. 바꿀 때마다 새 판본이 생깁니다. 그러니까 설계도는 하나여도, 고칠 때마다 그 설계도의 판본이 하나씩 쌓이는 거예요. 긴급 대응 프로세스의 정의에도 판본이 여럿 쌓여 있습니다. 왜 고치지 못하게 할까요? 지난 처리 건이 어떤 판본으로 돌았는지 나중에 되짚으려면, 그 판본이 그때 모습 그대로 남아 있어야 하니까요.

그런데 등록만 해서는 바뀌지 않아요. 둘째 칸, 경보를 받을 판본을 지정하는 배포를 해야 합니다. 배포는 어느 흐름이 그 경보를 받을지를 정하는 일이기도 해요. 같은 경보를 받는 흐름이 여러 개 배포돼 있으면, 가장 최근에 배포한 흐름이 경보를 받습니다. 쿨러 사건의 처리 건도 이렇게 배포된 긴급 대응 프로세스로 열렸어요. 배포된 흐름을 모두 거두면, HYD에 원래 들어 있는 기준 흐름

이 경보를 받고요. 셋째 실습에서 내 흐름을 배포하면, 가장 최근에 배포한 흐름이 되니까 쿨러 경보가 그때부터 내 흐름으로 열리게 됩니다.

셋째 칸, 배포한 뒤에 경보가 오면, 그 배포된 판본으로 처리 건이 열립니다.

마지막 칸이 중요해요. 이미 열려 있던 처리 건은, 자기가 열릴 때의 판본으로 끝까지 갑니다. 새 흐름을 배포했다고 진행 중인 건의 순서가 자동으로 바뀌지는 않아요. 다만 흐름 판본 고정이 센서 값이나 연결된 지식·서비스까지 얼린다는 뜻은 아닙니다. 승인할 때 현재 조건을 재확인하는 원칙은 그대로입니다. 고친 흐름이 다음 경보의 처리 건부터 쓰이는 까닭이 이겁니다.

[SLIDE 59] 감사 기록(Audit Log)과 처리 기록

마지막은 기록입니다. 프로세스가 실행되는 동안 일어난 일은 두 가지 기록으로 남아요. 표의 오른쪽 열이 그 구분입니다.

위 세 줄은 처리 기록이에요. 처리 건 한 건 안에서 일어난 일을 차례로 적은 겁니다. 승인이 접수될 때, 그 승인이 다음 단계로 전달될 때, 결과 보고가 나갈 때, 이렇게 한 줄씩 남아요. 승인 기록에는 승인한 역할과 판단 기록 번호가 담기고요. 그 번호 덕분에, 운전원이 어느 추천안을 보고 승인했는지, 그 추천안이 어떤 근거를 댔는지까지 거슬러 올라갈 수 있어요. 승인 전달 기록은 앞에서 본, 승인 내용이 다음 단계로 제대로 넘어갔는지 확인한 흔적이고요.

아래 세 줄은 감사 기록입니다. 누가 무엇을 결정했는지 나중에 확인하려고, 처리 건 밖의 일까지 적는 기록이에요. 처리 건이 열린 일, 정의의 판본이 배포된 일이 여기 남습니다.

맨 아래 줄을 보세요. 승인할 역할이 아닌 쪽이 승인을 시도했다가 거부된 일도 감사 기록에 남아요. 앞에서 프로세스가 책임을 보장한다고 했었죠. 그게 기록으로도 확인되는 겁니다. 그래서 끝난 처리 건 하나를 되짚을 때는, 상태와 판본에 더해서 이 두 기록을 함께 읽어요. 그러면 업무가 어떻게 끝났는지뿐 아니라, 누가 무엇을 결정했는지까지 확인할 수 있습니다.

[SLIDE 60~61] 실습: 간단한 승인 절차 작성

이제 이어질 네 실습을 순서대로 소개하겠습니다. 업무를 정의하고 그림으로 표현한 뒤 실행에 연결하고, 마지막으로 기록에서 결과를 확인하는 흐름이에요.

첫 실습에서는 작은 정비 요청 승인 절차를 그립니다. 웹 BPMN 편집기에서 요소를 놓고 연결하고, 이름을 붙여 저장한 뒤 다시 열어 볼 거예요. 요청과 담당자 확인, 승인 여부의 갈림을 그리고 승인 종료와 반려 종료를 구별합니다.

결과물은 BPMN 파일 하나입니다. 두 갈래가 각각 끝에 닿고 끊긴 선이나 이름 없는 갈래가 없는지 확인해요. 편집기 조작을 익히면서도, 핵심은 누가 무엇을 결정하고 각각 어디서 끝나는지를 다른 사람이 읽을 수 있게 만드는 데 있습니다.

[SLIDE 62~63] 실습: 긴급 대응 프로세스 작도

두 번째 실습에서는 작은 승인 절차에서 익힐 조작을 바탕으로 긴급 대응 프로세스를 그릴 겁니다. 지금까지 읽은 그림이 기준이고, 레인과 타이머가 더해집니다.

풀 하나를 담당자, 에이전트, 시스템의 세 레인으로 나누고 경보, 진단과 제안, 승인, 냉각 명령, 재관측, 작업지시와 보고를 배치합니다. 승인과 거절뿐 아니라 회복과 미달도 나눠요. 승인 작업에는 대기를 끊지 않고 알림 갈래를 여는 비중단형 타이머를 붙입니다.

결과물은 다음 연결 실습에서 쓸 BPMN 파일입니다. 완성 그림과 요소, 갈림, 타이머의 종류를 비교하고 각 경로를 따라가 보세요. 거절했는데 명령으로 이어지지 않는지, 알림이 끝났다고 승인까지 끝난 것으로 그리지 않았는지가 중요한 확인점입니다.

[SLIDE 64~65] 실습: 그린 흐름을 에이전트와 시스템에 연결

세 번째 실습에서는 그렇게 만들 파일을 실행 시스템에 연결합니다. BPMN에는 업무의 흐름이 있지만, 실제로 어떤 에이전트를 부르고 어느 계정이 승인하며 어떤 서비스를 실행할지는 더 정해야 하죠.

실습에서는 기존 냉각 긴급 대응 에이전트와 운전원 역할을 연결하고, 시작 경보, 시스템 작업, 분기 조건과 타이머 기한을 설정합니다. HYD 포털과 준비된 연결 자료를 사용하며, 화면에 없는 세부 연결은 제공 자료와 검증된 절차로 보완할 범위입니다.

연결을 마치면 사전 검사를 하고 문제를 고친 뒤 판본으로 등록하고 배포할 겁니다. 결과물은 등록된 정의와 경보를 받을 배포 상태예요. 사전 검사 통과는 정해진 정적 검사를 만족했다는 뜻입니다. 실제 서비스 접속과 실행 결과는 다음 실습에서 확인해야 합니다.

공용 HYD 환경에서는 같은 경보에 대해 마지막으로 배포한 흐름이 선택됩니다. 그래서 학생별 환경을 쓰거나 순서를 정해 진행하고, 실행하기 전에 어떤 흐름이 그 경보를 받는지 확인합니다.

[SLIDE 66~67] 실습: 설비 이벤트로 프로세스 실행

마지막 실습에서는 배포할 흐름이 실제 이벤트를 받아 어떻게 진행되는지 확인할 겁니다. 실습 환경의 쿨러 열화 기능으로 경보를 만들고, 내가 배포한 정의와 판본의 처리 건이 열렸는지 먼저 확인해요.

그다음 에이전트의 결과가 제출되고 운전원 승인을 기다리는 모습을 봅니다. 승인한 뒤에는 명령 처리와 재관측을 각각 확인하고, 어느 끝에 닿았는지와 결과 보고를 읽습니다. 기대한 회복이 확인되지 않으면 완료라는 상태만 보고 성공했다고 말하지 않습니다.

기준 흐름과도 구조를 비교할 거예요. 기준 흐름의 에이전트 단계 넷과 중단형 타이머, 수업 흐름의 에이전트 단계 하나와 비중단형 알림이 어떤 업무 차이를 만드는지 설명합니다. 두 흐름은 과거와 미래의 버전이라는 뜻은 아닙니다.

남길 것은 처리 건과 그 실행 기록입니다. 대기와 종료를 구별하고, 결과의 근거를 찾아 설명할 수 있어야 해요. 이제 네 실습을 차례로 진행한 뒤, 같은 이론의 복습 부분으로 돌아오겠습니다.

[SLIDE 68] 복습: 쿨러 사건 한 건을 프로세스로 읽기

실습에 이어 이번 회차를 정리하겠습니다. 처리 건에서 승인 대기와 끝난 상태를 구별해 보셨죠. 아직 기대한 결과를 얻지 못했다면, 어디까지 확인했고 무엇이 남았는지도 함께 적어 두세요. 그 눈으로 쿨러 사건 한 건을, 이번 회차에 배운 말로 다시 읽어 보겠습니다. 표 왼쪽의 장면마다 프로세스의 어떤 개념인지 먼저 말해 보세요. 그리고 오른쪽 열의 짝이 왜 서로 다른지 판단해 보세요.

첫 줄, 경보가 도착해서 처리 건이 열린 건, 메시지 시작 이벤트가 정의 하나에서 인스턴스 하나를 만든 거예요.

둘째 줄, 추천안이 나왔지만 팬은 그대로인 장면이죠. Task 하나가 끝난 순간입니다. End-to-End는 아직 진행 중이고요.

셋째 줄, 운전원 승인에서 기다리는 건 토큰이 사용자 작업에 머문 정상 상태입니다. 오류가 아니에요.

넷째 줄, 승인 여부 게이트웨이에서 승인 쪽으로 간 건 에이전트가 판단해서가 아닙니다. 운전원 승인이 낸 변수를 읽는 조건이 정한 거죠.

다섯째 줄, 운전원이 기한 안에 승인한 장면이에요. 승인 작업이 끝나면서 테두리의 비중단형 타이머도 함께 거둬졌습니다. 승인이 늦었다면 알림 가지만 열리고, 승인 작업은 그대로 기다렸겠죠. 알림이 나가는 것과 처리 건이 끝나는 것은 다른 일입니다.

마지막 줄, 쿨러 사건이 정상 보고로 끝난 건 끝 이벤트에 닿은 COMPLETED입니다. 그런데 거절로 닫힌 처리 건도 COMPLETED였죠. 성공인지 아닌지는 끝 이벤트와 결과 보고로 판단합니다.

여섯 장면이 전부 같은 그림, 같은 정의 위에서 일어났어요. 그림의 도형 하나하나가 실행에서 어떤 상태와 기록이 되는지 연결할 수 있으면, 처음 보는 처리 건도 읽어 낼 수 있습니다.

[SLIDE 69] 오늘 다룬 핵심 개념과 주요 용어 정리

사건의 흐름을 되짚었으니, 용어의 뜻도 한곳에 정리해 볼게요. 표에 아홉 개념의 이름과 뜻이 있습니다.

비즈니스 프로세스는 원하는 결과를 내기 위해 관련된 작업과 흐름을 정한 묶음이에요. BPM은 그 프로세스를 분석, 설계, 실행, 모니터링, 개선으로 되풀이하며 관리하는 방법이고요. BPMN은 도형과 선의 뜻을 정해서 업무 프로세스를 그리는 표기 표준입니다. 경계 이벤트는 작업 테두리에 붙어서, 그 작업이 진행 중일 때 생기는 일에 반응하는 이벤트고요.

프로세스 엔진은 프로세스 정의를 읽어서 작업을 차례로 열고 결과를 넘기는 소프트웨어예요. 이렇게 중앙의 조정자가 작업의 순서와 호출을 이끄는 방식을 오케스트레이션이라고 했죠. 프로세스 인스턴스는 정의에 따라 실제로 실행되는 한 건이고, HYD에서는 처리 건이라고 부릅니다. 토큰은

실행 중인 프로세스가 지금 어디에 있는지를 나타내는 표식이고요. 프로세스 변수는 인스턴스가지니고 다니는 이름 붙은 값입니다.

[SLIDE 70] Review Quiz: 핵심 흐름 되짚어보기

용어까지 정리했으니, 마지막으로 퀴즈를 풀어 보겠습니다. 세 문제로 배운 내용을 판단에 직접 써 볼 거예요. 용어를 외웠는지보다, 기록과 그림을 보고 무슨 일이 일어났는지 설명할 수 있는지가 중요합니다. 문제마다 답을 고를 시간을 드리고, 세 문항을 모두 본 다음에 정답과 까닭을 확인하겠습니다.

[SLIDE 71] Quiz 1. 승인 지연 알림이 나간 처리 건

첫 번째 문제예요. 긴급 대응 처리 건이 하나 있습니다. 상태는 실행 중이고, 열려 있는 단계는 운전원 승인이에요. 그런데 승인 지연 알림은 이미 나갔습니다. 이 처리 건에 대한 설명으로 옳은 건 무엇일까요?

1번, 알림이 나갔으므로 처리 건은 거절로 닫혔다. 2번, 기한이 지났으므로 냉각 명령이 자동으로 나갔다. 3번, 승인 작업이 열린 채로 남아 운전원의 결정을 기다린다. 4번, 알림 가지가 끝났으므로 처리 건도 완료되었다.

[SLIDE 72] Quiz 2. 설비 명령이 나갔는지를 판단하는 근거

두 번째 문제입니다. 어느 긴급 대응 처리 건의 기록에, 상태는 끝남, 닿은 끝 이벤트는 거절 종결이고, 냉각 명령 단계는 선택되지 않아 취소됐다고 적혀 있어요. 이 처리 건에서 냉각 명령이 설비에 나갔는지를 판단한 것으로 옳은 건 무엇일까요?

1번, 끝 이벤트가 거절 종결이고 냉각 명령 단계가 취소됨이므로 나가지 않았다. 2번, 상태가 끝남이므로 명령이 나가 유온이 회복되었다. 3번, 에이전트 단계가 끝났으므로 추천대로 명령이 나갔다. 4번, 기록으로는 알 수 없으므로 에이전트에게 다시 판단을 맡긴다.

[SLIDE 73] Quiz 3. 냉각 명령 작업을 여는 쪽

마지막 문제예요. 에이전트가 팬 최대 운전을 권하는 추천안을 제출했고, 처리 건은 운전원 승인에서 기다리고 있습니다. 운전원의 승인과 필요한 실행 조건 검사를 통과했을 때, 냉각 명령 작업을 여는 건 무엇일까요?

1번, 추천안을 제출한 에이전트. 2번, 명령을 받아 PLC에 전하는 SCADA. 3번, 승인한 운전원. 4번, 프로세스 엔진.

[SLIDE 74] 정답과 판단 기준

고른 답을 근거와 함께 맞춰 볼게요. 정답은 첫 문항 3번, 둘째 1번, 셋째 4번입니다.

첫 문항부터 볼게요. 승인 지연 타이머는 점선, 비중단형이었죠. 기한이 지나면 알림 가지만 열려서 그 가지만 끝나고, 승인 작업은 열린 채로 운전원을 기다립니다. 1번과 4번은 알림 가지의 끝을 처리 건 전체의 끝으로 헷갈린 거예요. 2번처럼 기한이 지났다고 자동으로 실행되는 일도 없습니다. 사람의 결정이 필요한 자리는 사람이 결정할 때까지 기다리니까요.

둘째 문항에서 끝남이라는 상태는 실행이 끝났다는 뜻일 뿐이에요. 명령이 나갔는지는 닿은 끝 이벤트와 냉각 명령 단계의 상태로 읽어야 합니다. 거절 종결에 닿았고 냉각 명령이 취소됨이니까, 명령은 나가지 않았어요. 2번은 끝남을 성공으로 읽은 거고, 3번은 추천과 실행을 같은 일로 본 거고, 4번은 이미 남아 있는 기록을 읽지 않은 겁니다.

셋째 문항에서 에이전트는 추천까지, 운전원은 승인까지 해요. SCADA는 열린 명령 작업의 명령을 PLC에 전하는 쪽이고요. 유효한 승인 결과와 필요한 조건을 확인하고 다음 작업인 냉각 명령을 여는 건 프로세스 엔진입니다. 그 작업이 열린 것과 실제 명령 처리가 성공한 것도 구별해야 합니다. 3번은 결정하는 쪽과 다음 작업을 여는 쪽을 헷갈린 거예요.

여기까지가 이번 회차입니다.

다음 회차에서는 같은 설계 원리를 설비 조치 밖으로 넓혀 볼 거예요. 정기 점검이 다가왔을 때, 그리고 예비품 재고가 부족할 때, 에이전트의 제안과 사람의 승인을 거쳐서 작업지시와 구매까지 이어지는 프로세스를 직접 만들고 실행해 보겠습니다. 이번 회차도 수고 많으셨습니다. 감사합니다.